Urdu Text-to-Speech System with Voice Cloning



By

Muhammad Bilal

B22F0382AI022

Rohail Ahmad

B22F1198AI014

Fiza Wahab Hanan Hakim

B22F0182AI026

Final Year Project Report

School of Computing Sciences

Pak-Austria Fachhochschule: Institute of Applied Sciences & Technology

Fall-22



Pak-Austria Fachhochschule:
Institute of Applied Sciences and Technology

Urdu Text-to-Speech System with Voice Cloning

A Final Year Project Report Presented to

Pak-Austria Fachhochschule:

Institute of Applied Sciences and Technology

In partial fulfillment

of the requirement for the degree of

BS Artificial Intelligence

By

Muhammad Bilal
B22F0382AI022

Rohail Ahmad
B22F1198AI014

Fiza Wahab Hanan Hakim
B22F0182AI026

Fall-22

Urdu Text-to-Speech System with Voice Cloning

A Final Year Project Report submitted to the School of School of Computing Sciences as partial fulfillment of the requirement for the award of Degree of BS in Artificial Intelligence.

Muhammad Bilal	B22F0382AI022
Rohail Ahmad	B22F1198AI014
Fiza Wahab Hanan Hakim	B22F0182AI026

Academic Supervisor

Dr. Arshad Iqbal
Assistant Professor,
School of Computing Sciences, SPCAI,
Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology

Industry Supervisor

Muhammad Murtaza Khan
Chief Innovation Officer at Sybrid Pvt. Ltd.

Final Approval

This final year project titles

Urdu Text-to-Speech System with Voice Cloning

By

Muhammad Bilal	*B22F0382AI022*
Rohail Ahmad	*B22F1198AI014*
Fiza Wahab Hanan Hakim	*B22F0182AI026*

Under the supervision of their project supervisor and approved by the project evaluation committee, has been accepted by the Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology, Pakistan, in partial fulfillment of the requirements for the degree of Program Name.

Academic Supervisor:

_____

Dr. Arshad Iqbal
School of Computing Sciences

Industry Supervisor:

_____

Muhammad Murtaza Khan
Chief Innovation Officer at Sybrid Pvt. Ltd.

HOD/ Chairman:

_____

Dr. Kamran Javed
School of Computing Sciences

Declaration

We, Muhammad Bilal, B22F0382AI022, Rohail Ahmad, B22F1198AI014, and Fiza Wahab Hanan Hakim, B22F0182AI026, hereby declare that we have produced the work presented in this final year project report, during the scheduled period of study. We also declare that we have not taken any material from any source except referred to wherever due, and that the amount of plagiarism is within an acceptable range. It is further declared that we have developed this project and the accompanied report entirely on the basis of our personal efforts made under the sincere guidance of our supervisor.

No portion of the work presented in this report has been submitted in support of any other degree or qualification at this or any other University or Institute of learning. If found otherwise, we shall stand responsible.

Date: _____

Signature: _____

Muhammad Bilal
B22F0382AI022

Signature: _____

Rohail Ahmad
B22F1198AI014

Signature: _____

Fiza Wahab Hanan Hakim
B22F0182AI026

Certificate

It is certified that Muhammad Bilal (B22F0382AI022), Rohail Ahmad (B22F1198AI014) and Fiza Wahab Hanan Hakim (B22F0182AI026) have carried out all the work related to this project under my supervision at the School of Computing Sciences, Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology and the work fulfills the requirement for the award of Artificial Intelligence.

Date: _____

Supervisor:

_____

Dr. Arshad Iqbal
Assistant Professor
School of Computing Sciences

Head of Department:

_____

Dr Kamran Javed
Associate Professor
School of Computing Sciences

DEDICATION

This thesis is lovingly dedicated to our parents, whose unwavering support, countless sacrifices, and constant prayers have been the cornerstone of our journey. Their belief in our potential has been a source of strength and inspiration at every step.

We also dedicate this work to our respected academic supervisor, Dr. Arshad Iqbal, for his invaluable guidance, patience, and mentorship, which have greatly contributed to the completion of this research.

Our sincere appreciation is extended to our industry supervisor, Dr. Muhammad Murtaza Khan of Sybrid Pvt. Ltd., whose practical insights and encouragement strengthened the real-world relevance of this work.

Finally, this work is dedicated to all Urdu speakers around the world, with the hope that this contribution plays a small role in advancing technology for our language and making it more accessible, expressive, and inclusive.

ACKNOWLEDGEMENTS

All praise and gratitude are first owed to Allah (SWT), the Most Gracious and Most Merciful, for granting us the knowledge, strength, and perseverance to complete this project.

We would like to express our sincere and deepest gratitude to our academic supervisor, Dr. Arshad Iqbal, Assistant Professor at the School of Computing Sciences, Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology. His patient guidance, constructive feedback, and constant encouragement throughout both semesters of this Final Year Project were invaluable in shaping the quality and direction of this work.

We are equally grateful to our industry supervisor, Muhammad Murtaza Khan, Chief Innovation Officer at Sybrid Pvt. Ltd., for providing real-world perspective, practical industry insights, and for expressing interest in evaluating our system for commercial deployment in call center automation. His engagement added significant motivation and purpose to our research.

We extend our sincere thanks to the faculty and staff of the School of Computing Sciences at PAF-IAST for their support throughout our academic journey. We also acknowledge the contributions of the open-source community, particularly the developers of ChatterBox TTS v0.1.2, the chatterbox-finetuning framework, and the Kaggle dataset contributors whose resources made this project technically feasible.

We are grateful to the ten participants who took time to evaluate our system during the user acceptance testing session. Their honest and enthusiastic feedback was both encouraging and validating.

Finally, we thank our families for their patience, prayers, and support during the long hours of development, training, and writing that this project demanded.

ABSTRACT

Urdu is one of the most widely spoken languages in South Asia, with over 170 million native speakers, yet it remains underrepresented in modern Text-to-Speech (TTS) technology. Existing Urdu TTS systems often produce robotic, unnatural speech, offer limited personalisation, and do not support zero-shot voice cloning, that is, synthesising speech in a target speaker's voice from only a short reference sample. This project addresses these limitations by designing, implementing, and evaluating بول (Bol) Urdu TTS, a web-based Urdu TTS system with integrated zero-shot voice cloning.

The system is built on ChatterBox TTS v0.1.2 (ResembleAI), a two-stage TTS model. Fine-tuning was performed using the Non-Turbo mode (IS_TURBO=False), which employs a compact Urdu-only BPE vocabulary of 2,454 tokens. The model was fine-tuned on 18,037 preprocessed Urdu audio samples selected from a 20,000-file Mozilla Common Voice dataset, using the chatterbox-finetuning framework on Google Colab with NVIDIA T4 GPU acceleration (CUDA 12.1). Fine-tuning completed in 3 hours, 9 minutes, and 22 seconds across 15 epochs and 6,525 optimisation steps, reducing training loss from 6.972 to 0.0637, a 99.1% decrease. Zero-shot voice cloning was implemented using the built-in Voice Encoder (VE) component of ChatterBox TTS v0.1.2, enabling personalised Urdu speech generation from a short reference audio sample without speaker-specific retraining.

The system was delivered as a full-stack web application comprising a React 19 frontend with Canvas 2D real-time audio visualisation (Web Audio API), a FastAPI backend with JWT-secured REST endpoints, and a SQLite database for user management and generation history. On 20 held-out test samples, the system achieved a Spectral Convergence of 1.4121, Log Spectral Distance of 1.0607, Pitch Error of 84.77 Hz, and an average GPU inference latency of 3.04 seconds per sentence. All 14 API unit tests and 9 non-functional requirement tests passed. Supervised user testing with 10 native Urdu-speaking participants yielded unanimously positive ratings for speech naturalness and intelligibility. The industry partner Sybrid has also expressed interest in evaluating the system for call center automation, underlining its commercial relevance

Keywords: Urdu Text-to-Speech, Voice Cloning, ChatterBox TTS v0.1.2, T3 Non-Turbo, Voice Encoder (VE), Fine-Tuning, Deep Learning, FastAPI, React 19, Natural Language Processing, Mozilla Common Voice.

Table of Contents

CHAPTER 1: INTRODUCTION 18

1.1 Problem Statement 18
1.2 Business Opportunity 18
1.3 Objectives 19
1.4 Project Scope 19
1.4.1 In-Scope 19
1.4.2 Out-of-Scope 20
1.5 Constraints 20
1.6 Feasibility 21
1.6.1 Technical Feasibility 21
1.6.2 Resource Feasibility 21
1.6.3 Time Feasibility 21
1.6.4 Market Feasibility 22
1.7 Stakeholders Description 22
1.7.1 Stakeholders Summary 22
1.7.2 Key High-Level Goals and Problems of Stakeholders 23

CHAPTER 2: LITERATURE REVIEW 24

2.1 Introduction 24
2.2 Review of Existing Systems 24
2.2.1 Early Statistical and Concatenative Systems 24
2.2.2 Attention-Based Neural TTS: Tacotron 24
2.2.3 Non-Autoregressive Models: FastSpeech 24
2.2.4 Multi-Speaker and Voice Cloning: YourTTS and XTTS 25
2.2.5 Massively Multilingual TTS: MMS-TTS 25
2.2.6 Flow-Matching Models: ChatterBox TTS v0.1.2 25
2.2.7 Urdu and Low-Resource South Asian Language TTS 26
2.3 Comparative Analysis of Techniques/Models 26
2.4 Voice Cloning Techniques 27
2.5 Evaluation Methodology in TTS Research 27
2.6 Summary 28

CHAPTER 3: SOFTWARE REQUIREMENT SPECIFICATION 29

3.1 Introduction 29
3.2 List of Features 29
3.3 Functional Requirements 30
3.4 Non-Functional Requirements 31
3.5 Use Cases / Use Case Diagram 32
3.5.1 Actors 32
3.5.2 Use Case List 33
3.5.3 Use Case Diagram 33
3.5.4 Key Use Case Descriptions 34
3.6 Software Development Plan 36
3.6.1 Development Methodology 36
3.6.2 Development Phases 36
3.6.3 Technology Stack 37

CHAPTER 4: SYSTEM OVERVIEW 39

4.1 Introduction 39
4.2 Architectural Design 39

4.2.1 Three-Tier Architecture *39*
4.3 ChatterBox TTS v0.1.2 Pipeline 40
4.4 Data Design 41
4.4.1 Entity-Relationship Diagram (ERD) *41*
4.4.2 Data Flow Diagram (DFD), Level 0: Context Diagram *43*
4.4.3 Data Flow Diagram, Level 1: Main Processes *44*
4.5 Domain Model 45
4.5.1 Class Diagram *45*
4.6 Behaviour Models 47
4.6.1 Activity Diagram *47*
4.6.2 Class-Level Sequence Diagram, TTS Generation *48*
4.6.3 Component Diagram *48*
4.7 System Mockups 49
4.8 Methodology and System Workflow 55
5.1 Introduction 56
5.2 Technology Stack (Languages, Frameworks, Tools) 56
5.3 Development Environment Setup 56
5.3.1 Training Environment (Google Colab) *56*
5.3.2 Backend Environment *57*
5.3.3 Frontend Environment *57*
5.4 System Modules and Components 57
5.5 Database Implementation 58
5.6 APIs and External Integrations 58
5.7 Dataset Selection and Preprocessing 59
5.7.1 Dataset Acquisition *59*
5.7.2 Audio Preprocessing *59*
5.8 Model Selection and Training 61
5.8.1 Model Selection Rationale *61*
5.8.2 Fine-Tuning Configuration *61*
5.8.3 Training Process *63*
5.8.4 Alignment Stream Analyzer Fix *64*
5.9 Model Evaluation 65
5.10 Model Deployment 65

CHAPTER 6: SYSTEM TESTING AND DEPLOYMENT 66

6.1 Introduction 66
6.2 Testing Approach 66
6.3 Unit Testing 66
6.4 Integration Testing 68
6.5 System Testing 68
6.6 User Acceptance Testing (UAT) 68
6.7 Performance Testing 69
6.8 Deployment Strategy 71
6.9 Hosting and Deployment Environment 72
6.10 Version Control and CI/CD Pipeline 72

CHAPTER 7: RESULTS AND DISCUSSION 74

7.1 Introduction 74
7.2 Evaluation and Results 74
7.2.1 ML Model Evaluation Results *74*
7.2.2 Comparison with Initial Objectives *78*
7.2.3 User Feedback and Usability Testing *79*
7.2.4 Consolidated Performance Summary *80*

7.3 Challenges and Limitations 81

CHAPTER 8: CONCLUSION AND FUTURE WORK **83**

8.1 Conclusion 83
8.2 Future Enhancements 84
8.2.1 Extended Model Training *84*
8.2.2 Formal MOS Evaluation *84*
8.2.3 Automated WER Evaluation *84*
8.2.4 GPU-Enabled Production Deployment *84*
8.2.5 Expressive and Emotional Speech Synthesis *85*
8.2.6 Native Mobile Application *85*
8.2.7 Multilingual South Asian Extension *85*

BIBLIOGRAPHY **86**

List of Figures

Figure 1 Use Case Diagram 34
Figure 2 System Architecture 40
Figure 3 ChatterBox TTS v0.1.2 internal pipeline 41
Figure 4 Entity Relationship Diagram 43
Figure 5 Data Flow diagram level 0 44
Figure 6 Data Flow diagram level 1 45
Figure 7 Class Diagram: Urdu TTS System (Backend and Frontend classes with attributes, methods, and relationships) 46
Figure 8 Activity diagram 47
Figure 9 Sequence diagram 48
Figure 10 Component Diagram 49
Figure 11 Home Page 50
Figure 12 Features Page: Detailed Capability Breakdown 51
Figure 13 About Page: Team Member Cards and Supervisors 52
Figure 14 Login Page and Register Page 53
Figure 15 Dashboard: TTS Generation Tab with Canvas 2D Audio Visualiser 54
Figure 16 Dashboard: Voice Cloning Tab with Reference Audio Upload Interface 54
Figure 17 Dashboard: History Tab Showing Past Generation Records 55
Figure 18: Training Loss Convergence Curve (Epochs 1-15) 64
Figure 19: ChatterBox Urdu TTS: ML Model Performance Metrics Dashboard 77
Figure 20: ChatterBox Urdu TTS: Comprehensive Evaluation Dashboard 78

List of Tables

Table 1 Stakeholders Summary 22
Table 2 Comparative Analysis of TTS Systems 26
Table 3 Features List 29
Table 4 Functional Requirements 30
Table 5 Non-Functional Requirements 32
Table 6 Use Case Descriptions 35
Table 7 Project Timeline 36
Table 8 Technology Stack 37
Table 9 Users Table Schema 42
Table 10 Generations Table Schema 42
Table 11 API Endpoints 58
Table 12 Dataset Preprocessing Summary 60
Table 13 Dataset Statistics 60
Table 14 Fine-Tuning Configuration Parameters 61
Table 15 Training Loss Progression (Key Epochs) 63
Table 16 API Endpoint Unit Test Results 67
Table 17 User Acceptance Testing Summary 69
Table 18 Non-Functional Requirements Test Results 70
Table 19 Deployment Architecture 71
Table 20 Version Control and CI/CD Configuration 73
Table 21 Twenty Urdu Test Sentences Used for Formal Evaluation 74
Table 22 Objective ML Evaluation Metrics (20 Test Samples) 75
Table 23 Training Performance Metrics 76
Table 24 Project Objectives vs. Achieved Outcomes 79
Table 25 Consolidated System Performance Summary 80

List of Acronyms

Acronym	Full Form
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
ASR	Automatic Speech Recognition
BPE	Byte Pair Encoding
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CUDA	Compute Unified Device Architecture
DB	Database
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
EOS	End-of-Sequence
F0	Fundamental Frequency
GPU	Graphics Processing Unit
HF	Hugging Face
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
LSD	Log Spectral Distance
MCD	Mel-Cepstral Distortion
ML	Machine Learning

MOS	Mean Opinion Score
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PAF-IAST	Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology
RMS	Root Mean Square
REST	Representational State Transfer
RTF	Real-Time Factor
SPA	Single-Page Application
SQL	Structured Query Language
TTS	Text-to-Speech
UAT	User Acceptance Testing
VAD	Voice Activity Detection
VE	Voice Encoder
VRAM	Video Random Access Memory
WAV	Waveform Audio File Format
WER	Word Error Rate
WebGL	Web Graphics Library

Chapter 1: Introduction

1.1 Problem Statement

Urdu, spoken natively by over 170 million people across South Asia and diaspora communities worldwide, remains severely underserved by modern Text-to-Speech (TTS) technology. Existing commercial and open-source TTS systems either do not support Urdu at all, or produce robotic, monotone speech that is clearly distinguishable from natural human voice. None of the widely available systems support personalised voice cloning for Urdu, that is, the ability to generate Urdu speech that sounds like a specific person from a short audio reference.

This gap has real consequences. Visually impaired Urdu speakers cannot access digital content through natural-sounding assistive technology. Educational platforms cannot deliver engaging audio content in Urdu. Call centres serving Pakistan's large Urdu-speaking population must rely on either low-quality automated voices or expensive human agents. The absence of a high-quality, personalisable Urdu TTS system represents both a technological gap and a social one.

This project addresses the gap by fine-tuning ChatterBox TTS v0.1.2, a two-stage TTS model, on 18,037 preprocessed Urdu audio samples selected from a 20,000-file Mozilla Common Voice dataset. It integrates zero-shot voice cloning through the model's built-in Voice Encoder (VE) and delivers the complete system as a deployed, publicly accessible web application named بول (Bol), meaning 'Speak' in Urdu.

1.2 Business Opportunity

Developing a high-quality Urdu TTS system with voice cloning presents a substantial business opportunity. Pakistan's digital economy is growing rapidly, with over 120 million internet users and a strong push toward digitised government services, e-learning, and customer service automation. All of these domains require natural-sounding Urdu speech.

Few existing products offer high-quality Urdu TTS, and none combine it with voice cloning. The system developed in this project, بول (Bol, meaning 'Speak'), fills this gap. Sybrid, the industry partner for this project, has expressed interest in evaluating بول for customer service automation in call centres, which provides direct commercial validation. Potential application areas include audiobooks and educational content narration, accessibility tools for visually impaired Urdu

speakers, virtual assistants and chatbots interacting in natural personalised Urdu, call centre automation with consistent branded voices, and content creation tools enabling podcast producers to generate Urdu voiceovers at scale.

1.3 Objectives

The project has the following specific objectives:

(i) To build a natural-sounding Urdu TTS system by fine-tuning ChatterBox TTS v0.1.2 on 18,037 preprocessed Urdu audio samples selected from a 20,000-file Mozilla Common Voice Urdu dataset.

(ii) To integrate zero-shot voice cloning so that users can generate Urdu speech in any voice by providing a short reference audio sample, using the built-in Voice Encoder (VE) component of ChatterBox TTS v0.1.2.

(iii) To develop a full-stack web application, بول Urdu TTS, comprising a React 19 frontend with Canvas 2D real-time audio visualisation (Web Audio API) and a FastAPI backend with JWT authentication and generation history.

(iv) To deploy the complete system publicly at zero cost using Hugging Face Spaces (backend) and Vercel (frontend), making it accessible to Urdu-speaking users worldwide.

(v) To evaluate system quality through objective metrics (training loss, spectral measures) and subjective user acceptance testing with native Urdu-speaking participants.

1.4 Project Scope

1.4.1 In-Scope

Urdu TTS Model: Fine-tuning ChatterBox TTS v0.1.2 using Non-Turbo mode (IS_TURBO=False) with a 2,454-token Urdu-only BPE vocabulary on 18,037 valid audio samples from the Mozilla Common Voice Urdu dataset to produce natural Urdu speech at 24,000 Hz sample rate in WAV format.

Voice Cloning: Zero-shot voice cloning using the integrated Voice Encoder (VE) component of ChatterBox TTS v0.1.2, conditioned on a 3-second reference audio prompt.

Web Application: A deployed React 19 SPA with sidebar dashboard, RTL Urdu text input, Canvas 2D audio visualisation (Web Audio API), generation history, and dark/light theme support.

Backend API: A FastAPI backend with JWT-based authentication (HS256, 7-day expiry), TTS inference, voice cloning, and SQLite-backed history management.

CI/CD Pipeline: GitHub Actions automating deployment of the frontend to Vercel and the backend to Hugging Face Spaces on every push to the main branch.

Evaluation: Objective metric evaluation on 20 held-out test samples and a supervised user acceptance testing session with native Urdu speakers.

1.4.2 Out-of-Scope

Multilingual Support: The system targets Urdu exclusively in this phase. Extension to other languages is designated as future work.

Emotional and Expressive Speech: Prosody control for emotional tone (happy, sad, angry) is excluded from this phase.

Native Mobile Application: A dedicated iOS or Android app is excluded; the responsive web application serves mobile browsers.

Long-term Post-deployment Maintenance: Ongoing support and updates beyond the FYP submission period are not included.

1.5 Constraints

Dataset Availability: High-quality, labelled Urdu speech data is limited. The Mozilla Common Voice Urdu corpus (~20,000 raw clips, 18,037 valid after preprocessing) is one of the few large-scale freely available Urdu speech datasets.

Compute Resources: Full model training requires a GPU. This constraint is addressed by using Google Colab T4 GPU for fine-tuning and Hugging Face Spaces for inference.

Deployment Compute: The production HF Spaces deployment runs on CPU free tier, giving an average inference time of approximately 25 seconds per sentence. GPU deployment would reduce this to under 3 seconds.

Academic Timeline: All development, training, and evaluation must be completed within two academic semesters (FYP-1 and FYP-2).

Budget: The project must be implemented at zero financial cost, using exclusively free-tier cloud services and open-source tools.

1.6 Feasibility

1.6.1 Technical Feasibility

ChatterBox TTS v0.1.2, released by ResembleAI in 2025, provides a strong open-source technical foundation. Its two-stage architecture separates text-to-token alignment (T3 stage) from waveform synthesis (S3Gen stage), meaning only the T3 stage requires retraining for a new language. The Non-Turbo mode uses a compact Urdu-only BPE vocabulary of 2,454 tokens, small enough to train efficiently on 18,037 Urdu samples. The S3Gen decoder and Voice Encoder (VE) are frozen throughout training, significantly reducing training time and resource requirements. The fine-tuning framework (github.com/gokhaneraslan/chatterbox-finetuning) provides a ready-made training pipeline accepting standard LJSpeech-format datasets.

1.6.2 Resource Feasibility

The project requires no paid software licences or proprietary APIs. Google Colab T4 GPU provides free fine-tuning compute. Hugging Face Hub provides up to 50 GB of free model storage. Hugging Face Spaces provides free Docker-based backend hosting. Vercel provides free CDN hosting for the React 19 frontend. The three-person development team brings complementary expertise in AI and model engineering, full-stack web development, and data processing.

1.6.3 Time Feasibility

The project is structured across two semesters. FYP-1 covers literature review, dataset acquisition and preprocessing, system design, and initial model experiments. FYP-2 covers fine-tuning, full-

stack development, CI/CD pipeline setup, deployment, evaluation, and documentation. This timeline is adequate for all planned deliverables given the availability of pre-built training infrastructure.

1.6.4 Market Feasibility

The global Urdu-speaking population exceeds 230 million when second-language speakers are included. Pakistan alone has approximately 120 million internet users, a growing e-commerce sector, and an expanding digital services industry that increasingly requires Urdu voice interfaces. The absence of competitive high-quality Urdu TTS with voice cloning means early entrants face minimal direct competition. Sybrid's interest in the system for call centre automation provides concrete evidence of commercial demand.

1.7 Stakeholders Description

1.7.1 Stakeholders Summary

Table 1 identifies the key stakeholders of the بول Urdu TTS project alongside their roles and responsibilities. These range from the end users who generate and clone Urdu speech, to the development team responsible for building and fine-tuning the system, the academic supervisor providing technical guidance, and the industry partner (Sybrid) evaluating the system for call-centre automation. Identifying these stakeholders at the outset ensured that the functional and non-functional requirements defined in the following sections reflected the needs of every party involved.

Table 1 Stakeholders Summary

Stakeholder	Role	Responsibilities
Muhammad Bilal, Rohail Ahmad, Fiza Wahab Hanan Hakim	Development Team	Design, development, training, and testing of the Urdu TTS system.
Dr. Arshad Iqbal	Academic Supervisor	Provides academic guidance and ensures research alignment with institutional standards.
Muhammad Murtaza Khan (Sybrid)	Industry Supervisor	Provides industry insights and ensures the project meets commercial standards.
Urdu-speaking individuals and organisations	End Users	Benefit from natural-sounding Urdu speech synthesis and voice cloning features.

Sybrid	Industry Partner	Potential adopter of the system for call center automation and customer service.
PAF-IAST	Academic Institution	Oversees FYP completion and assessment standards.

1.7.2 Key High-Level Goals and Problems of Stakeholders

Development Team: The primary goal is to deliver a complete, deployable Urdu TTS system with zero-shot voice cloning. Key challenges include managing limited Urdu speech data, achieving perceptually natural speech through fine-tuning of ChatterBox TTS v0.1.2, and completing all phases within the academic timeline.

Academic Supervisor: The goal is to ensure the project meets the research and documentation standards of PAF-IAST, with particular attention to rigorous evaluation methodology and comprehensive academic reporting.

Industry Supervisor (Sybrid): The goal is to assess whether بول Urdu TTS is suitable for integration into call centre workflows. Their key concern is speech naturalness and the reliability of the deployed API.

End Users: End users seek Urdu speech that is clearly intelligible, natural-sounding, and if desired, in their own voice. Their primary concern is that the system works reliably for everyday Urdu text without producing robotic or garbled output.

Sybrid (Industry Partner): As a potential commercial adopter, Sybrid's goal is a system that can be integrated into existing call centre infrastructure with smooth API integration and consistent output quality at scale.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews existing work in TTS synthesis, focusing on neural end-to-end models, voice cloning techniques, and prior work on Urdu and other low-resource South Asian languages. The review explains why ChatterBox TTS v0.1.2 was chosen as the core model for this project and identifies the research gaps that بول Urdu TTS addresses.

2.2 Review of Existing Systems

2.2.1 Early Statistical and Concatenative Systems

The first generation of TTS systems used concatenative synthesis, which stitched together pre-recorded units of speech stored in a large database. Systems such as Festival [25] and MBROLA [26] achieved reasonable intelligibility but produced speech with audible concatenation artefacts. Statistical parametric synthesis using Hidden Markov Models (HMMs), pioneered by the HTS system [21], replaced hand-crafted rule sets with learned acoustic models but produced characteristically buzzy speech. Neither approach was well-suited to low-resource languages such as Urdu, as both required extensive language-specific engineering.

2.2.2 Attention-Based Neural TTS: Tacotron

The Tacotron model [4] introduced the first fully neural, end-to-end TTS architecture trained directly from text-audio pairs without phoneme dictionaries or hand-crafted features. Tacotron uses a sequence-to-sequence model with Bahdanau attention to learn text-to-mel-spectrogram alignment, followed by the WaveNet vocoder for waveform synthesis. Tacotron 2 [5] improved naturalness, reaching a mean opinion score of 4.53, close to that of recorded human speech. However, it required hours of single-speaker training data and was not designed for multi-speaker voice cloning.

2.2.3 Non-Autoregressive Models: FastSpeech

FastSpeech [6] addressed the slow autoregressive inference of Tacotron by introducing a parallel, non-autoregressive decoder guided by a duration predictor. FastSpeech 2 [7] extended the approach by taking pitch, energy, and duration directly from the speech signal as conditioning inputs,

improving prosody control. Non-autoregressive models are suitable for low-latency inference but typically produce less natural prosody variation when extrapolating to new languages.

2.2.4 Multi-Speaker and Voice Cloning: YourTTS and XTTS

YourTTS [9] extended the VITS architecture [8] to zero-shot multi-speaker synthesis by conditioning the model on speaker embeddings extracted from a reference audio sample, demonstrating voice cloning from a few seconds of audio. However, YourTTS was evaluated mainly on English and Portuguese, and its performance on Urdu and the Nastaliq script was not demonstrated. XTTS [27], which builds on the Tortoise model, extended zero-shot synthesis to sixteen languages but does not natively support Urdu, and its large size makes fine-tuning computationally expensive.

2.2.5 Massively Multilingual TTS: MMS-TTS

Meta's Massively Multilingual Speech (MMS) project [10] trained speech synthesis models covering 1,107 languages with the VITS architecture, including Urdu. Its per-language acoustic quality is lower than a dedicated single-language model because model capacity is shared across so many languages. More importantly for this project, MMS-TTS does not support voice cloning; it produces only a fixed, pre-trained Urdu voice.

2.2.6 Flow-Matching Models: ChatterBox TTS v0.1.2

ChatterBox TTS v0.1.2 [1] is a recent open-source neural TTS model. It has a two-stage architecture: a T3 stage (Text-to-Tokens Transformer) followed by an S3Gen stage (a Speech-to-Spectrogram Generator based on flow-matching). This split separates the language-modelling problem from the acoustic synthesis problem. The T3 stage is a LLaMA-style autoregressive transformer that converts text tokens into discrete speech tokens. The S3Gen stage then uses flow-matching to decode those tokens into mel spectrograms and, finally, into 24,000 Hz waveforms.

Importantly for this project, ChatterBox TTS includes a built-in Voice Encoder (VE) that extracts speaker embeddings from a 3-second reference audio sample, which enables zero-shot voice cloning without any speaker-specific retraining. The open-source chatterbox-finetuning repository [2] allows fine-tuning of the T3 stage only, in Non-Turbo mode with a compact 2,454-token Urdu-only

BPE vocabulary, while keeping the S3Gen decoder and Voice Encoder frozen. This greatly reduces the training data and compute required for a new language.

2.2.7 Urdu and Low-Resource South Asian Language TTS

Research on Urdu TTS has been limited by the scarcity of publicly available labelled Urdu speech corpora. Early Urdu TTS systems [17] were rule-based concatenative systems with a constrained vocabulary. A more recent survey [18] of Urdu ASR and TTS set out the main challenges: the right-to-left Nastaliq script, the heavy use of diacritical marks (harakat), and the small size of publicly available training corpora. The Mozilla Common Voice project provides approximately 20,000 validated Urdu audio clips, from which 18,037 valid samples were selected after preprocessing for this project. To the best knowledge of the project team, no prior published work has applied ChatterBox TTS fine-tuning to Urdu, making بول Urdu TTS a novel contribution.

2.3 Comparative Analysis of Techniques/Models

Table 2 Comparative Analysis of TTS Systems

System	Architecture	Urdu Support	Voice Cloning	Open Source	Suitability
Tacotron 2	Seq2Seq + WaveNet	None	No	Yes	Low: single speaker, no cloning
FastSpeech 2	Non-autoregressive	None	No	Yes	Low: no cloning
YourTTS	VITS + speaker embed	None	Yes (zero-shot)	Yes	Medium: no Urdu
XTTS-v2	GPT-2 + flow-match	None	Yes (zero-shot)	Yes	Medium: compute heavy
MMS-TTS	VITS (multilingual)	Yes (basic)	No	Yes	Low: fixed voice, no cloning
ChatterBox v0.1.2	T3 + S3Gen (flow)	Via fine-tuning	Yes (VE, 3s ref)	Yes	HIGH: selected for this project

As Table 2 shows, ChatterBox TTS v0.1.2 is the best-positioned option for this project: it supports zero-shot voice cloning natively through its built-in Voice Encoder (VE), is fully open-source with an available fine-tuning framework, and its two-stage Non-Turbo architecture allows language adaptation of the T3 stage alone, without retraining the whole model. No other reviewed system meets all four needs at once: Urdu support through fine-tuning, zero-shot voice cloning, open-source availability, and efficient fine-tuning on limited data.

2.4 Voice Cloning Techniques

Voice cloning in TTS refers to synthesising speech that perceptually matches a target speaker's voice characteristics, such as pitch, timbre, speaking rate, and prosody, from a short reference audio sample, without retraining the model on that speaker's data. The common approach uses a speaker encoder network trained on a large multi-speaker corpus. At synthesis time, the encoder extracts a fixed-dimensional speaker embedding from the reference audio, and this embedding conditions the TTS decoder. The Generalised End-to-End (GE2E) loss [11] is a widely used method for training robust speaker encoders. ChatterBox TTS uses ResembleAI's built-in Voice Encoder (VE) to extract speaker embeddings from 3-second reference clips, conditioning the T3 transformer to produce speech tokens that reflect the reference speaker's characteristics.

2.5 Evaluation Methodology in TTS Research

Evaluation of TTS systems employs both objective and subjective metrics. The Mean Opinion Score (MOS) is the primary subjective metric where human listeners rate speech naturalness on a five-point scale. A MOS above 4.0 is considered near-human quality. Objective metrics include Mel Cepstral Distortion (MCD), which measures spectral envelope similarity; Word Error Rate (WER), measured by an ASR system; and Real-Time Factor (RTF). As discussed in Chapter 7, MCD values are heavily influenced by cross-speaker evaluation methodology: comparing synthesised speech against a different speaker's reference produces MCD values many times higher than same-speaker comparison. This project therefore uses training loss convergence, spectral metrics, inference time, and a supervised UAT session with ten native Urdu-speaking listeners as its primary evaluation framework.

2.6 Summary

This chapter reviewed the evolution of TTS from concatenative systems through attention-based neural models to modern flow-matching-based architectures. It established that ChatterBox TTS v0.1.2 is the most suitable base model for an Urdu TTS system with voice cloning within the resource constraints of this project. The literature confirmed that Urdu TTS remains an understudied problem. The following chapter presents the software requirements specification.

Chapter 3: Software Requirement Specification

3.1 Introduction

This chapter presents the complete Software Requirements Specification (SRS) for the بول Urdu TTS system. It defines all functional requirements, non-functional requirements, the feature list, use case descriptions and diagrams, and the software development plan. All requirements are grounded in the actual implemented system: a ChatterBox TTS v0.1.2 fine-tuned model served through a FastAPI backend and accessed via a React 19 frontend deployed on Vercel and Hugging Face Spaces.

3.2 List of Features

Table 3 presents the complete set of features offered by the بول Urdu TTS system. Each feature is assigned an identifier (F1 to F10), a short description, and a priority level that guided the order of implementation. High-priority features such as Urdu speech generation and zero-shot voice cloning form the core of the system, while lower-priority features such as generation history and audio download improve usability. This prioritisation ensured that the most critical capabilities were built and tested first.

Table 3 Features List

F#	Feature	Description	Priority
F-01	Urdu TTS Synthesis	Generate Urdu speech from text input using fine-tuned ChatterBox TTS v0.1.2 at 24 kHz WAV output.	High
F-02	Zero-Shot Voice Cloning	Clone any voice from a 10-30s reference audio sample using the built-in Voice Encoder (VE) component.	High
F-03	Canvas 2D Audio Visualiser	Breathing sphere that pulses with real-time frequency data from the Web Audio API (AnalyserNode, fftSize=64).	Medium
F-04	User Authentication	JWT-based (HS256, 7-day expiry) registration and login with bcrypt password hashing.	High
F-05	Generation History	SQLite-backed history of all past TTS and voice cloning generations per user with replay control.	High

F-06	Audio Download	Download generated speech as 24 kHz mono WAV files for offline use.	Medium
F-07	Dark / Light Theme	Theme toggle persisted in localStorage, shared via ThemeContext across public pages and dashboard.	Low
F-08	Responsive Web Design	Responsive layout for desktop and mobile browsers using custom CSS variables.	Medium
F-09	Background Generation	TTS and clone requests continue running when the user navigates between dashboard tabs via GenerationContext.	Medium
F-10	Microphone Recording	In-browser microphone capture for voice cloning reference audio using the MediaRecorder API.	Medium

3.3 Functional Requirements

Building on the feature list, Table 4 specifies the functional requirements, that is, the concrete behaviours the software must exhibit. Each requirement (FR1 to FR10) maps to one or more features and describes a specific action the system performs, such as authenticating users, accepting Urdu text, generating speech, accepting a reference sample for cloning, and storing generation history. These requirements later served as the checklist against which the system was verified during testing in Chapter 6.

Table 4 Functional Requirements

FR#	Feature	Description
FR-1	User Registration	Users create an account with username, email, and password. Passwords are hashed using bcrypt (bcrypt==4.0.1 with passlib compatibility patch).
FR-2	User Login & JWT Auth	Users log in with valid credentials and receive an HS256 JWT access token (7-day expiry). Token stored in localStorage key 'token'.
FR-3	Simple TTS Generation	Authenticated users input Urdu text (up to ~200 characters / 256 tokens) via an RTL text area and POST to /tts_simple to receive a WAV audio file.

FR-4	Voice Cloning TTS	Users upload or record a reference audio (WAV preferred, MP3 auto-converted). Backend POSTs to /clone. Voice Encoder (VE) extracts speaker embedding from the first 3 seconds.
FR-5	Audio Playback with Visualiser	Frontend plays generated WAV using the HTML5 Audio API. Web Audio AnalyserNode (fftSize=64) feeds a Canvas 2D breathing sphere visualiser.
FR-6	Audio Download	Users can download generated speech as a WAV file (24 kHz, mono) directly from the audio player.
FR-7	Generation History	GET /history retrieves all past generations for the authenticated user ordered by timestamp, showing text preview, model type badge, and replay control.
FR-8	Account Management	Users can view and update their profile via GET/PUT /users/me and toggle dark/light theme from the Settings tab.
FR-9	Background Generation	GenerationContext lifts TTS/clone fetch state above route components so navigating between dashboard tabs never cancels an in-progress request.
FR-10	API Health Check	GET /health returns server and model status without authentication, confirming ChatterBox TTS is loaded and ready.

3.4 Non-Functional Requirements

In contrast to the functional requirements, Table 5 defines the non-functional requirements, which describe how well the system must perform rather than what it must do. The requirements are grouped by category, including performance, usability, security, and reliability, and specify measurable targets, for example that a sentence should be synthesised within ten seconds and that all protected routes must enforce JWT authentication. These targets provided the quantitative benchmarks used to evaluate the deployed system in Section 6.7.

Table 5 Non-Functional Requirements

NFR#	Category	Requirement
NFR-01	Performance	TTS generation must complete within 10 seconds per sentence on GPU-enabled hardware (measured: 3.04s avg on Colab T4).
NFR-02	Audio Quality	Generated speech must be clearly intelligible and natural-sounding to native Urdu speakers (confirmed: 10/10 UAT positive).
NFR-03	Security	All user data protected by HS256 JWT authentication. Passwords hashed with bcrypt. All protected endpoints verify Bearer tokens. HTTPS enforced in production.
NFR-04	Usability	Web interface must be intuitive for non-technical Urdu-speaking users with RTL text input support, responsive design, and dark/light mode.
NFR-05	Scalability	FastAPI backend handles concurrent requests using async endpoints with Uvicorn ASGI server.
NFR-06	Reliability	System must handle invalid input gracefully, returning proper HTTP error codes (400, 401, 422) with meaningful messages.
NFR-07	Maintainability	Code follows modular three-tier architecture: /frontend (React 19), /hf-space backend (FastAPI), /t3_finetuned_model training scripts.
NFR-08	Portability	Backend deployable as Docker container on any CPU or CUDA-enabled host. Frontend deployable to any static CDN via Vite build output.
NFR-09	Zero Cost Deployment	Entire production stack must be hosted on free-tier platforms (Hugging Face Spaces + Vercel) with total monthly cost of PKR 0.

3.5 Use Cases / Use Case Diagram

3.5.1 Actors

Three actors interact with the بول Urdu TTS system:

Registered User: An authenticated Urdu-speaking individual who accesses TTS generation, voice cloning, history, and settings features via the web dashboard after logging in with a valid JWT token.

Guest (Unauthenticated): A visitor who can browse the public pages (Home, Features, About) but must register or log in before accessing any TTS or voice cloning functionality.

System: The backend TTS pipeline, comprising the ChatterBox TTS v0.1.2 inference engine (T3 + S3Gen + Voice Encoder VE), the FastAPI REST API server, and the SQLite database.

3.5.2 Use Case List

The system supports eleven use cases grouped into three functional areas: (1) Account and UI management: Register / Sign Up, Login, Manage Account, Change Theme. (2) TTS and voice functionality: Input Urdu Text, Generate Speech (TTS), Record or Upload Reference Audio, Voice Cloning. (3) Output management: Playback Audio, Download Audio (WAV), View Generation History.

3.5.3 Use Case Diagram

Figure 1 presents the use case diagram for the system, giving a visual overview of how the two actors, the User and the System interact with the available use cases. The User performs the use cases of registering and logging in, generating Urdu speech, cloning a voice from a reference sample, and viewing their generation history, while the System actor carries out the underlying speech synthesis and voice-cloning processing in response to those requests. Modelling these interactions clarified the boundary between user-initiated actions and system responsibilities, which directly informed the authentication and authorisation logic implemented in the backend.

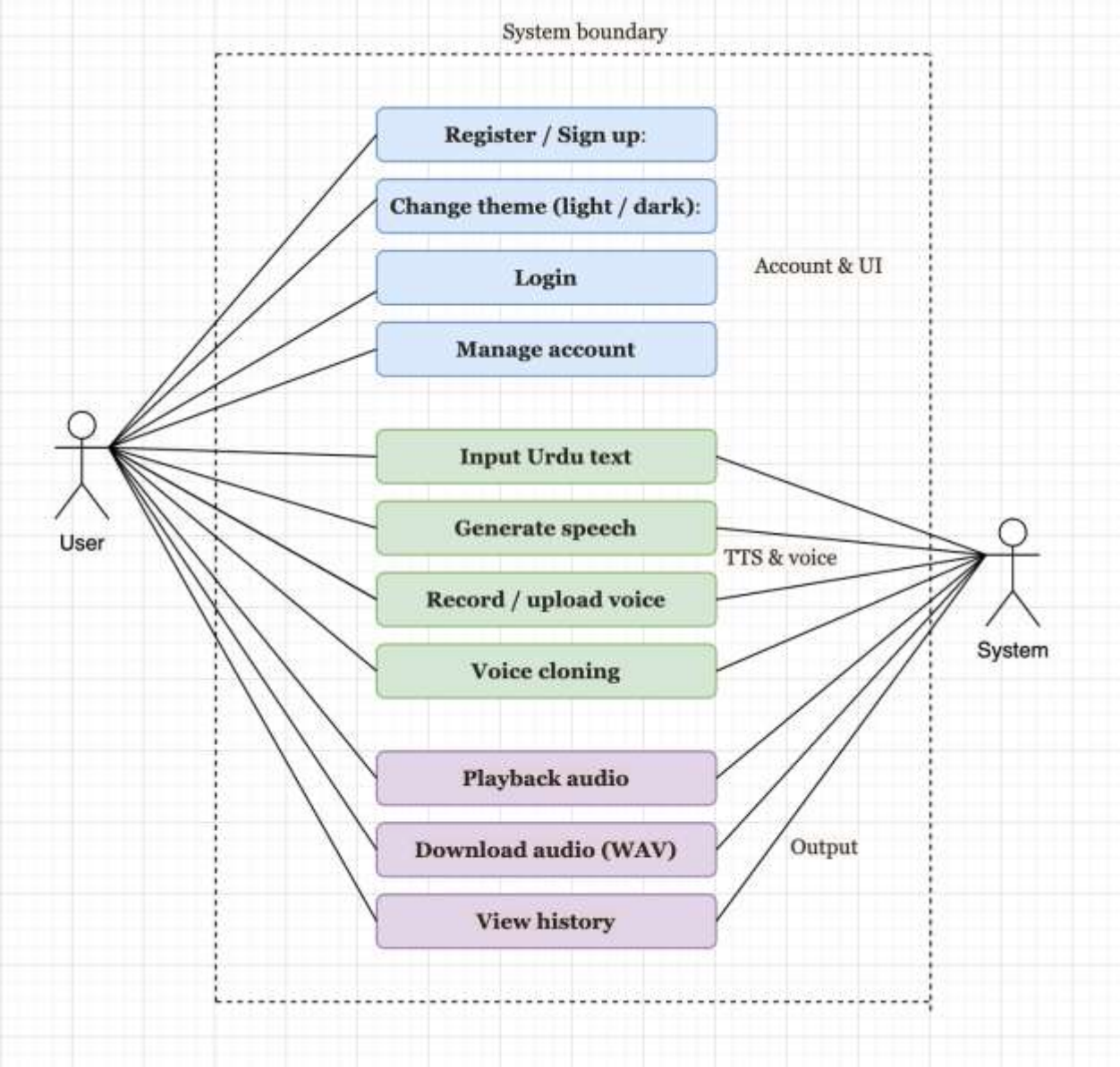


Figure 1 Use Case Diagram

3.5.4 Key Use Case Descriptions

Table 6 expands the two most important use cases from the diagram, UC-01 (Generate Urdu Speech) and UC-02 (Clone Voice and Generate Speech), into detailed descriptions. For each use case the table specifies the primary actor, preconditions, the main flow of events, and the expected outcome, giving a step-by-step account of how a user moves from entering Urdu text or uploading a reference sample to receiving synthesised audio. These descriptions bridge the high-level requirements and the concrete behaviour implemented in Chapter 5.

Table 6 Use Case Descriptions

Field	UC-01: Generate Urdu Speech
Actor	Registered User
Precondition	User is logged in; valid JWT token stored in localStorage.
Main Flow	1. User navigates to TTS Dashboard tab. 2. User enters Urdu text (RTL, up to ~200 chars). 3. User clicks Generate Speech. 4. Frontend POSTs {text} to /tts_simple with Bearer token. 5. Backend validates token, splits text on sentence boundaries. 6. T3 generates speech tokens; S3Gen decodes to waveform; VAD trims silence. 7. Segments concatenated with 0.2s pauses; 24 kHz WAV returned. 8. Audio plays with Canvas 2D visualiser. 9. Record written to history table.
Postcondition	WAV audio generated, played, and stored in generation history.
Exception	Empty or non-Urdu text returns HTTP 400. Expired JWT returns HTTP 401.

Field	UC-02: Clone Voice and Generate Speech
Actor	Registered User
Precondition	User is logged in and has a reference audio sample (WAV preferred, 10-30 seconds).
Main Flow	1. User navigates to Voice Cloning tab. 2. User uploads audio file or records via microphone (MediaRecorder API). 3. Frontend auto-converts non-WAV audio to WAV. 4. User enters Urdu text and sets tau slider (0.1-1.0). 5. Frontend POSTs multipart form to /clone. 6. Voice Encoder (VE) extracts speaker embedding from first 3s of reference. 7. Embedding conditions T3 to generate speaker-specific speech tokens. 8. S3Gen decodes tokens to personalised WAV. 9. Audio plays; stored in history as model_type=clone.
Postcondition	Personalised WAV generated, played, and stored in history with 'clone' badge.
Exception	Reference audio shorter than 3s may reduce cloning quality. Non-audio files return HTTP 422.

3.6 Software Development Plan

3.6.1 Development Methodology

The project follows an Agile development methodology with iterative sprints. Each sprint covers one functional layer of the system, with integration testing at the end of each sprint. This approach allows early detection of integration issues and accommodates the iterative nature of ML model development, where training results inform subsequent data and configuration decisions.

3.6.2 Development Phases

Table 7 sets out the project timeline across the two semesters of the final year project. It divides the work into sequential phases, from requirement gathering and dataset preparation, through model fine-tuning and web-application development, to testing and deployment, listing the duration and key activities of each. This phased plan reflects the iterative methodology described above and shows how the substantial machine-learning and full-stack components were scheduled within the available time.

Table 7 Project Timeline

Phase	Semester	Duration	Key Activities
Requirements & Research	FYP-1	Month 1	Literature review, TTS model evaluation, dataset identification, requirement gathering.
System Design	FYP-1	Month 2	Three-tier architecture design, database schema, API contract, UI mockups.
Dataset Preparation	FYP-1	Month 3	Mozilla Common Voice Urdu download, LJSpeech conversion, VAD preprocessing, metadata generation.
Model Research & Baseline	FYP-1	Month 4	ChatterBox TTS v0.1.2 baseline inference, tokeniser analysis, fine-tuning framework setup.
Model Fine-Tuning	FYP-2	Month 5	T3 stage fine-tuning on Colab T4 GPU, 15 epochs, loss monitoring, weight export to HF Hub.
Backend Development	FYP-2	Month 6	FastAPI endpoints, JWT auth, SQLite ORM, inference integration, Docker containerisation.

Frontend Development	FYP-2	Month 6	React 19 SPA, GenerationContext, ThemeContext, Canvas 2D visualiser, RTL text input.
Integration & CI/CD	FYP-2	Month 7	GitHub Actions workflows, HF Spaces backend deploy, Vercel frontend deploy, CORS configuration.
Testing & Evaluation	FYP-2	Month 7	Unit tests (14 cases), integration tests, UAT session (10 Urdu speakers), metric evaluation.
Documentation & Demo	FYP-2	Month 8	FYP report finalisation, Colab GPU demo notebook, defence preparation.

3.6.3 Technology Stack

Table 8 documents the complete technology stack used to build بول Urdu TTS, organised by category. It lists the machine-learning components (ChatterBox TTS v0.1.2, PyTorch, and librosa), the web-application stack (React 19 and Vite on the frontend, FastAPI and SQLite on the backend), and the supporting infrastructure (Google Colab for training, Hugging Face Spaces and Vercel for hosting), together with the specific version of each. Recording exact versions makes the system reproducible, which matters given the rapid pace of change in the underlying libraries.

Table 8 Technology Stack

Category	Technology	Version / Details
TTS Core Model	ChatterBox TTS	v0.1.2, two-stage T3 + S3Gen architecture (ResembleAI)
TTS Stage 1	T3 Transformer (Non-Turbo)	LLaMA-based autoregressive transformer; fine-tuned on Urdu (IS_TURBO=False)
TTS Stage 2	S3Gen Decoder	Flow-matching decoder; frozen during Urdu fine-tuning
Voice Cloning	Voice Encoder (VE)	Built-in speaker embedding extractor (part of ChatterBox TTS v0.1.2); 3s reference prompt
Fine-tuning Mode	Non-Turbo (IS_TURBO=False)	Urdu-only BPE vocabulary: 2,454 tokens (vs 52,260 Turbo base)

Deep Learning	PyTorch + TorchAudio	v2.6.0 with CUDA 12.1
Fine-tuning Framework	chatterbox-finetuning	github.com/gokhaneraslan/chatterbox-finetuning
Backend Framework	FastAPI + Uvicorn	Async REST API with ASGI server; Python 3.11
Authentication	JWT (HS256) / OAuth2 Bearer	python-jose, passlib with bcrypt==4.0.1
Database	SQLite + SQLAlchemy	ORM-based; persistent on HF Spaces /data volume
Frontend Framework	React 19 + Vite	v19.2.0 / v7.2.4, component-based SPA
Routing	React Router DOM	v7.13.0, client-side SPA routing
Audio Visualiser	Canvas 2D + Web Audio API	Breathing sphere; AnalyserNode fftSize=64
Styling	Custom CSS + CSS Variables	Navy blue design system; dark mode via data-theme attribute
Icons	lucide-react	v0.563.0
Training Platform	Google Colab T4 GPU	CUDA 12.1; Micromamba tts environment (Python 3.10)
Model Storage	Hugging Face Hub	mbilalcr07/bol-urdu-tts, ~5 GB total weights
Backend Hosting	Hugging Face Spaces	mbilalcr07/bol-backend, Docker, CPU free tier
Frontend Hosting	Vercel CDN	voicegen-frontend.vercel.app, auto-deploy from GitHub
CI/CD	GitHub Actions	Bilal-Shinwari/FYP-Project, dual workflow: HF + Vercel
Dataset	Mozilla Common Voice (Urdu)	~20,000 raw clips; 18,037 valid after preprocessing; 24 kHz
Version Control	Git + GitHub	github.com/Bilal-Shinwari/FYP-Project

Chapter 4: System Overview

4.1 Introduction

This chapter presents the complete architectural and design overview of the بول Urdu TTS system. It describes the three-tier system architecture, the ChatterBox TTS v0.1.2 inference pipeline, data flow diagrams, the entity-relationship diagram, class diagram, sequence diagrams, activity diagram, component diagram, state transition diagram, domain model, and the database design. All diagrams reflect the actual deployed system.

4.2 Architectural Design

4.2.1 Three-Tier Architecture

The بول Urdu TTS system follows a three-tier client-server architecture, shown in Figure 2. All communication between tiers is via RESTful HTTP over HTTPS, secured with JWT Bearer tokens on all protected routes.

The Frontend tier is a React 19 Single Page Application hosted on Vercel's global CDN at voicegen-frontend.vercel.app. It handles all user interaction including Urdu text input (RTL), audio playback with the Canvas 2D breathing sphere visualiser, voice cloning file upload or microphone recording, and generation history display. State is managed through three React Contexts: AuthContext (JWT storage), ThemeContext (dark/light mode), and GenerationContext (in-progress TTS/clone requests).

The Backend API tier is a FastAPI application running in a Docker container on Hugging Face Spaces (mbilalcr07-bol-backend.hf.space). It exposes eight REST endpoints, validates JWT tokens, manages the SQLite database persisted on the /data volume, and orchestrates TTS inference. The model is loaded once at server startup via snapshot_download from Hugging Face Hub and held in memory.

The ML Inference Engine tier consists of the fine-tuned ChatterBox TTS v0.1.2 model weights (~5 GB total): the fine-tuned t3_finetuned.safetensors (~2 GB) and the frozen pretrained components including ve.safetensors, s3gen.safetensors, t3_cfg.safetensors, tokenizer.json, and conds.pt (~3

GB). Inference runs on CPU in the production HF Spaces environment (~25 seconds per sentence) and on GPU in the Colab demo environment (~3 seconds per sentence on T4).

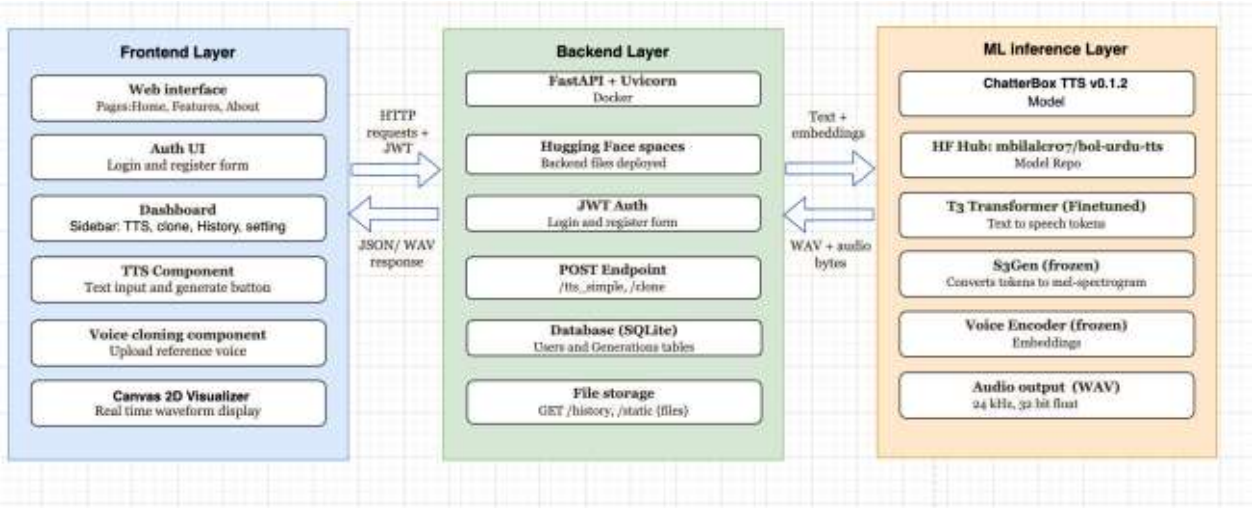


Figure 2 System Architecture

4.3 ChatterBox TTS v0.1.2 Pipeline

The internal two-stage inference pipeline of ChatterBox TTS v0.1.2, illustrated in Figure 3, transforms Urdu text into a 24 kHz WAV audio file as follows.

Stage 1, the T3 stage (Text-to-Tokens Transformer): The backend preprocesses input Urdu text by removing diacritical marks (harakat) and splitting the text on sentence boundaries. Each sentence is tokenised using the custom Urdu BPE tokeniser (vocabulary size 2,454, Non-Turbo mode). The T3 LLaMA-based autoregressive transformer generates discrete speech tokens from the text token sequence. An Alignment Stream Analyzer hooks into attention layers and monitors text-speech alignment to detect and suppress token repetition loops by forcing early End-of-Sequence (EOS). The EOS threshold was fixed to 4 consecutive identical tokens after a minimum of 30 generation steps, specifically tuned for Urdu phoneme patterns (see Section 5.8.4).

Stage 2, the S3Gen stage (Speech Synthesis Generator): The generated speech token sequence is decoded by the S3Gen flow-matching decoder into a mel spectrogram and then into a raw 24 kHz audio waveform. If a reference audio is provided (voice cloning mode), the Voice Encoder (VE)

extracts a speaker embedding from the first 3 seconds, conditioning the T3 transformer to generate speech tokens reflecting the reference speaker's vocal characteristics.

Post-processing: Voice Activity Detection (VAD) trims silence from each sentence segment. Segments are concatenated with 0.2-second padding. The final output is a 24 kHz mono WAV file returned to the frontend via the REST API.

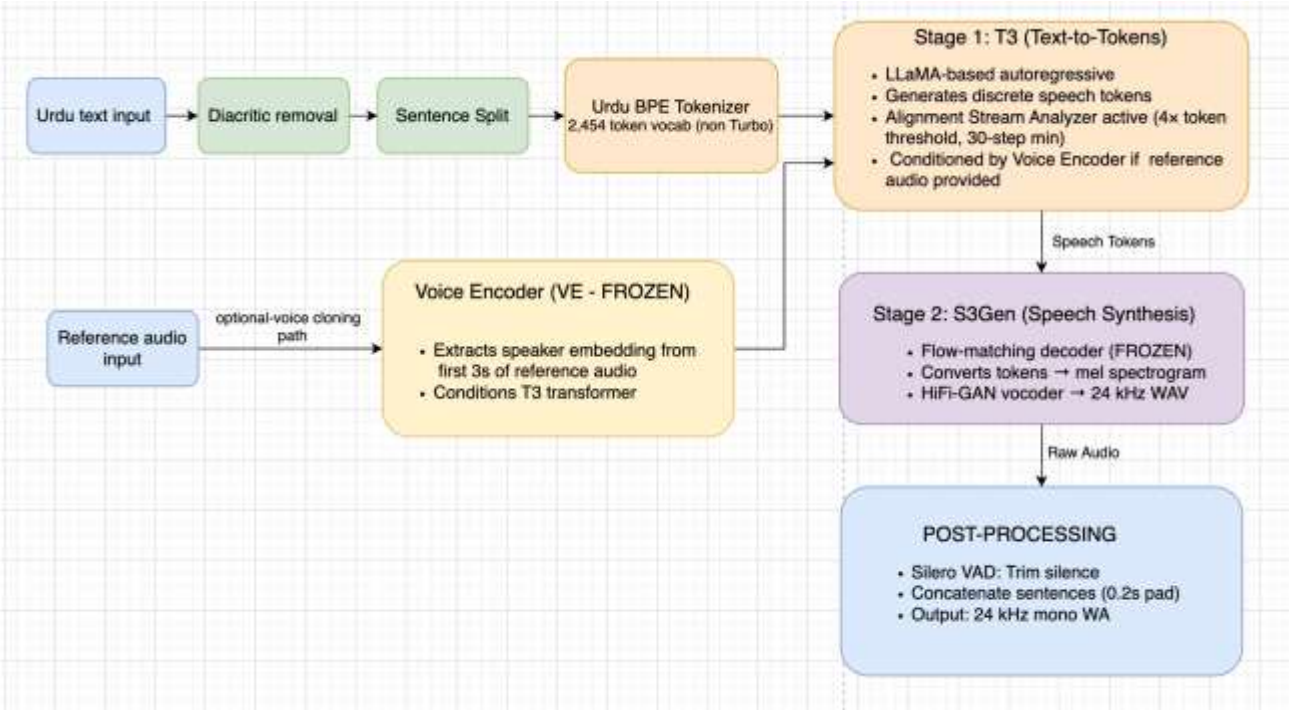


Figure 3 ChatterBox TTS v0.1.2 internal pipeline

4.4 Data Design

4.4.1 Entity-Relationship Diagram (ERD)

The SQLite database managed by SQLAlchemy ORM contains two entities, User and History, whose schemas are given in Table 9 and Table 10. The relationship is one-to-many: one User can have zero or many History records linked via the user_id foreign key, as shown in the entity-relationship diagram in Figure 4.

Table 9 Users Table Schema

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AutoIncrement	Unique user identifier
username	VARCHAR	NOT NULL, UNIQUE	User's chosen display name
email	VARCHAR	NOT NULL, UNIQUE	User's email address for login
hashed_password	VARCHAR	NOT NULL	bcrypt-hashed password (bcrypt==4.0.1)
is_active	BOOLEAN	DEFAULT TRUE	Account active status flag

Table 10 Generations Table Schema

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AutoIncrement	Unique history record identifier
user_id	INTEGER	FOREIGN KEY → User.id	Owner of this generation record
input_text	TEXT	NOT NULL	Urdu text input used for generation
audio_path	VARCHAR	NOT NULL	File path under /data/static/ for generated WAV
model_type	VARCHAR	NOT NULL	Generation type: 'TTS' or 'clone'
created_at	DATETIME	NOT NULL	Timestamp of generation (UTC)

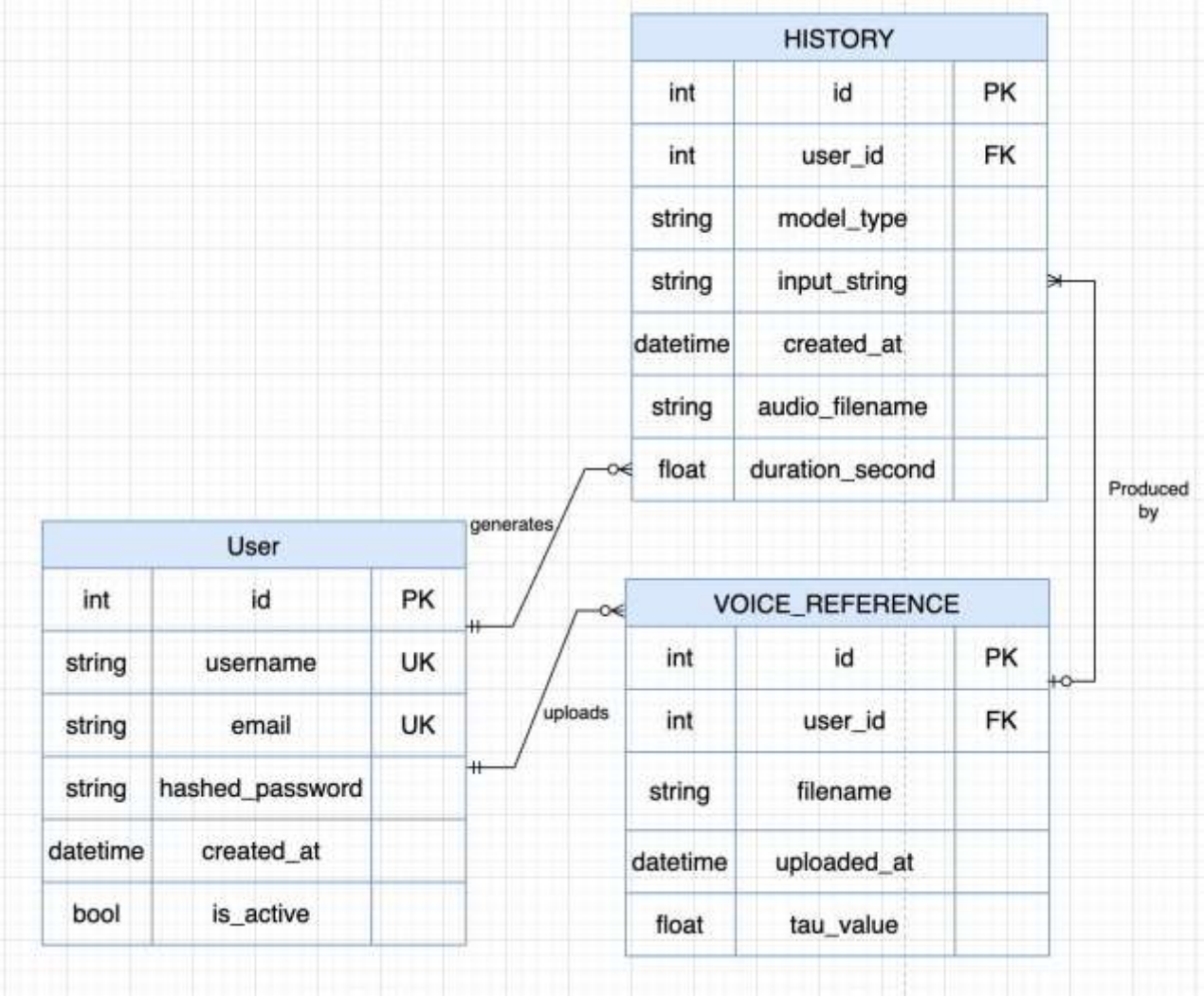


Figure 4 Entity Relationship Diagram

4.4.2 Data Flow Diagram (DFD), Level 0: Context Diagram

The Level 0 DFD, shown in Figure 5, represents the entire بول Urdu TTS system as a single process. The Registered User provides Urdu text input and an optional reference audio file, and receives generated WAV audio in return. The Training Dataset (Mozilla Common Voice Urdu) was consumed offline during the fine-tuning phase and is not an active runtime entity.

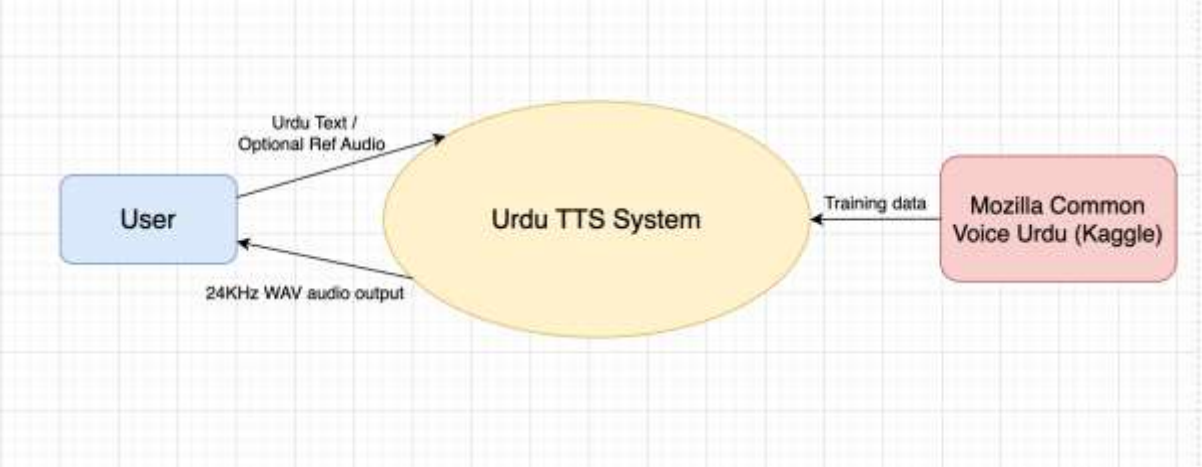


Figure 5 Data Flow diagram level 0

4.4.3 Data Flow Diagram, Level 1: Main Processes

The Level 1 DFD, shown in Figure 6, decomposes the system into four main processes: P1 (User Management) handles registration, login, JWT issuance, and profile management; P2 (TTS Processing) receives authenticated text and runs the ChatterBox TTS v0.1.2 pipeline; P3 (Voice Cloning) receives authenticated text plus reference audio, extracts the speaker embedding via the Voice Encoder (VE), and generates personalised WAV output; P4 (History Management) reads from the Generations data store and the Static Files store.

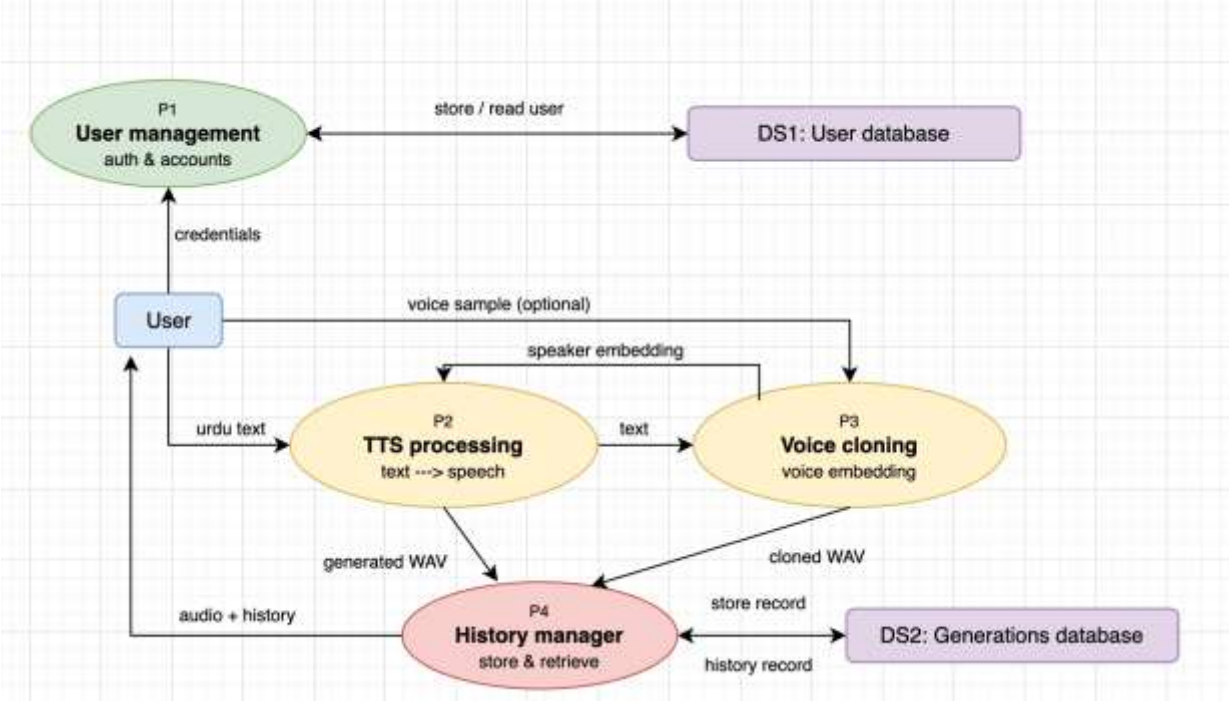


Figure 6 Data Flow diagram level 1

4.5 Domain Model

4.5.1 Class Diagram

The class diagram in Figure 7 shows the main software classes across the three tiers. Backend classes: User (id, username, email, hashed_password, is_active), History (id, user_id, input_text, audio_path, model_type, created_at), TTSEngine (model, base_params; methods: load_finetuned_engine(), generate(), clone()), AuthHandler (secret_key, algorithm; methods: hash_password, verify_password, create_token, decode_token). Frontend classes: AuthContext (token, isAuthenticated), GenerationContext (isGenerating, audioUrl), ThemeContext (theme), AudioVisualizer (analyser: AnalyserNode), DashboardLayout (sidebar with pulsing generation indicator).

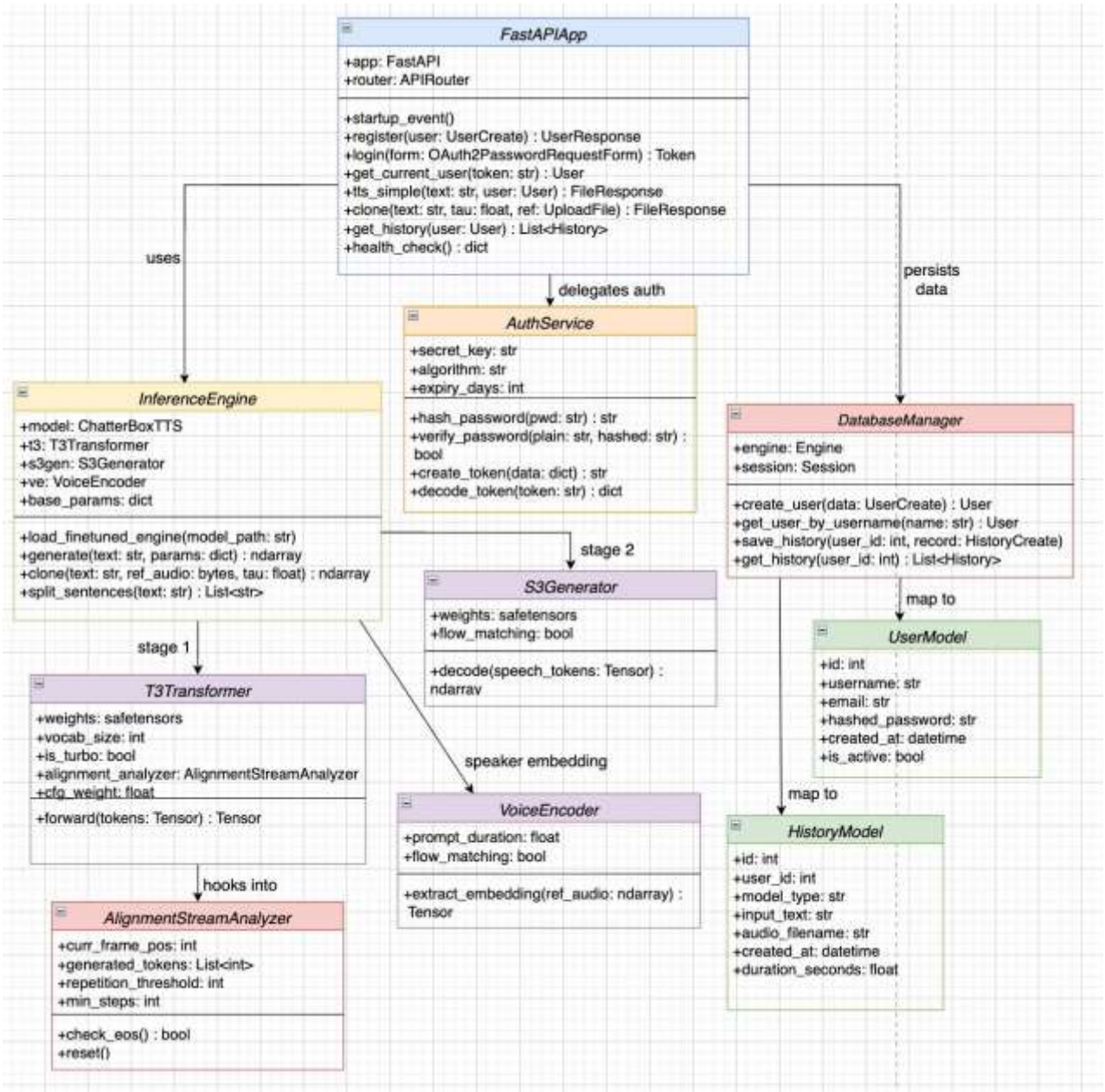


Figure 7 Class Diagram: Urdu TTS System (Backend and Frontend classes with attributes, methods, and relationships)

4.6 Behaviour Models

4.6.1 Activity Diagram

The activity diagram in Figure 8 shows the complete user activity flow from landing on the application through authentication, text entry, the TTS or voice cloning decision branch, speech generation, playback, and optional download.

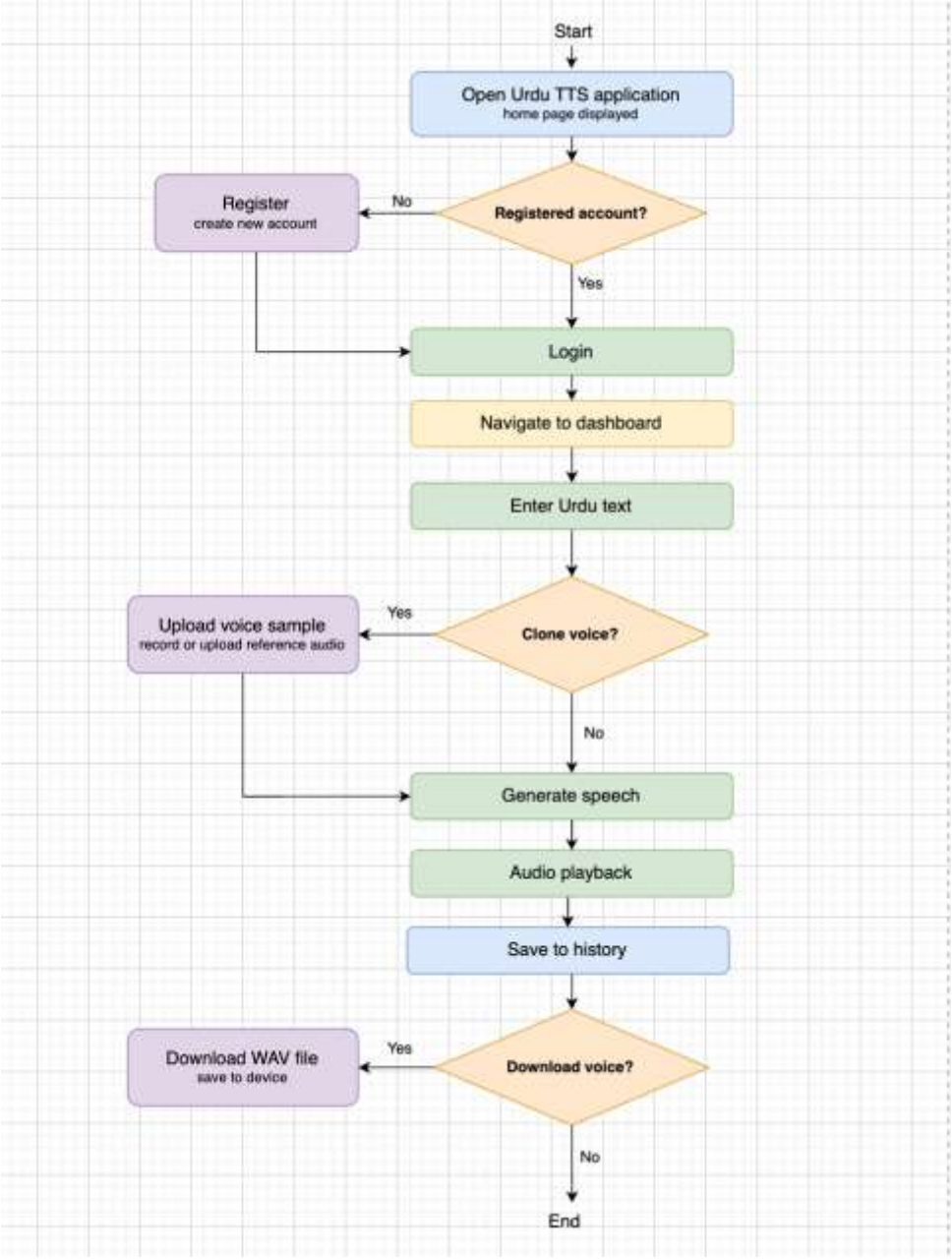


Figure 8 Activity diagram

4.6.2 Class-Level Sequence Diagram, TTS Generation

The sequence diagram for TTS generation (Figure 9) traces the message flow from the user clicking Generate Speech to receiving the WAV audio, passing through GenerationContext, FastAPI backend, AuthHandler, and the TTSEngine (T3 + S3Gen + VAD).

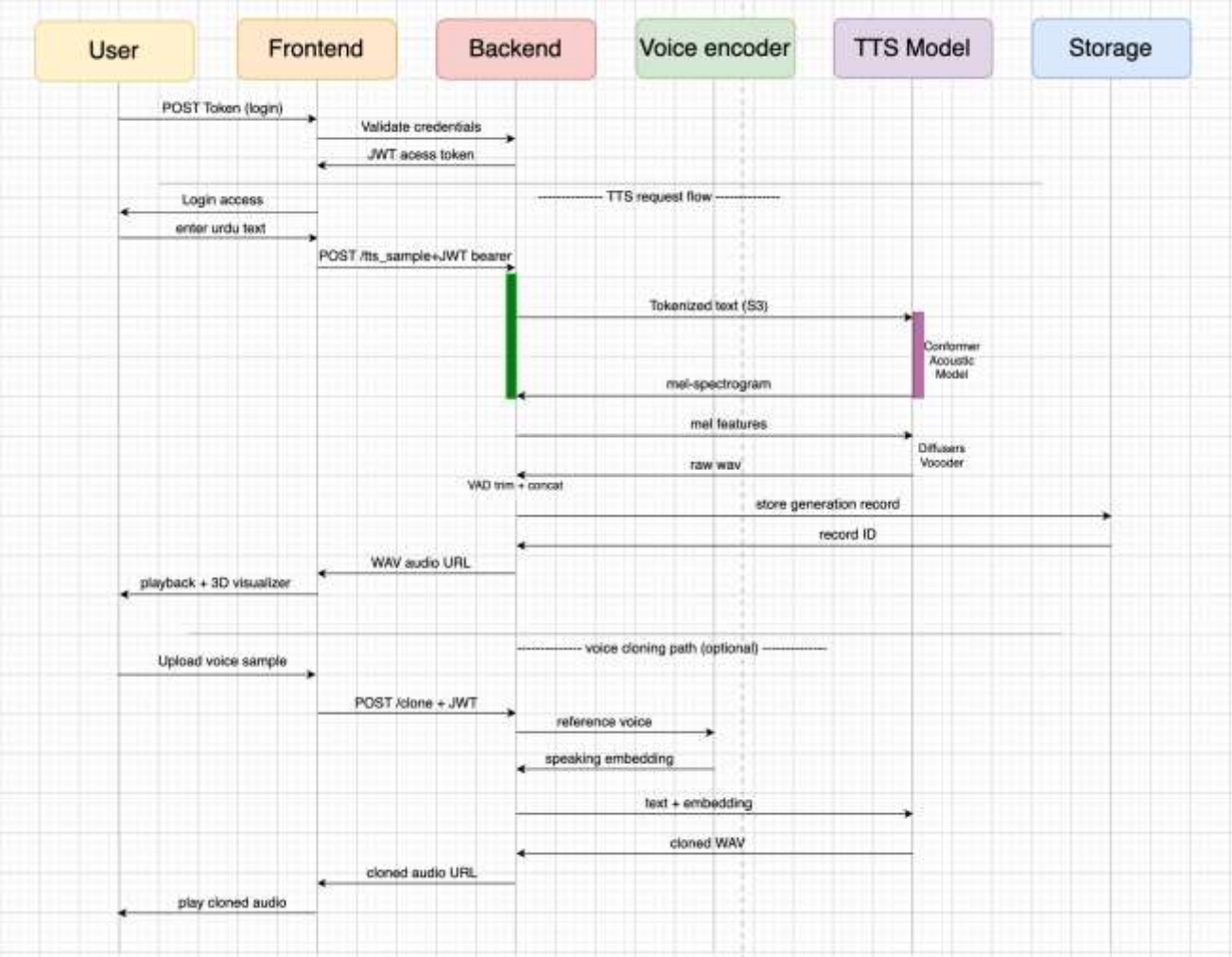


Figure 9 Sequence diagram

4.6.3 Component Diagram

The component diagram in Figure 10 shows the modular structure of the deployed system organised into three packages: the React 19 Frontend (Vercel), the FastAPI Backend (HF Spaces Docker), and the ML Inference Layer (HF Hub model weights), with CI/CD automated via GitHub Actions.

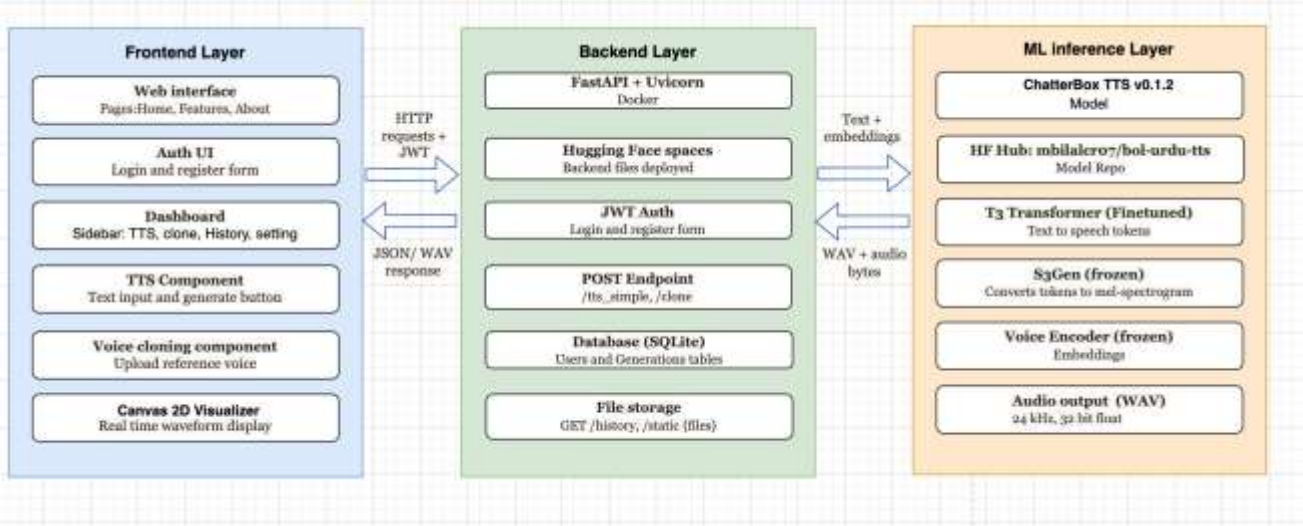


Figure 10 Component Diagram

4.7 System Mockups

The following figures show the live web application of بول Urdu TTS, taken from the deployment at voicegen-frontend.vercel.app. Figure 11 shows the home page, Figure 12 the features page, and Figure 13 the about page with the team and supervisor cards. Figure 14 shows the login and registration pages. Figure 15 shows the TTS generation tab with the Canvas 2D audio visualiser, Figure 16 the voice cloning tab with its reference-audio upload interface, and Figure 17 the history tab listing past generations.

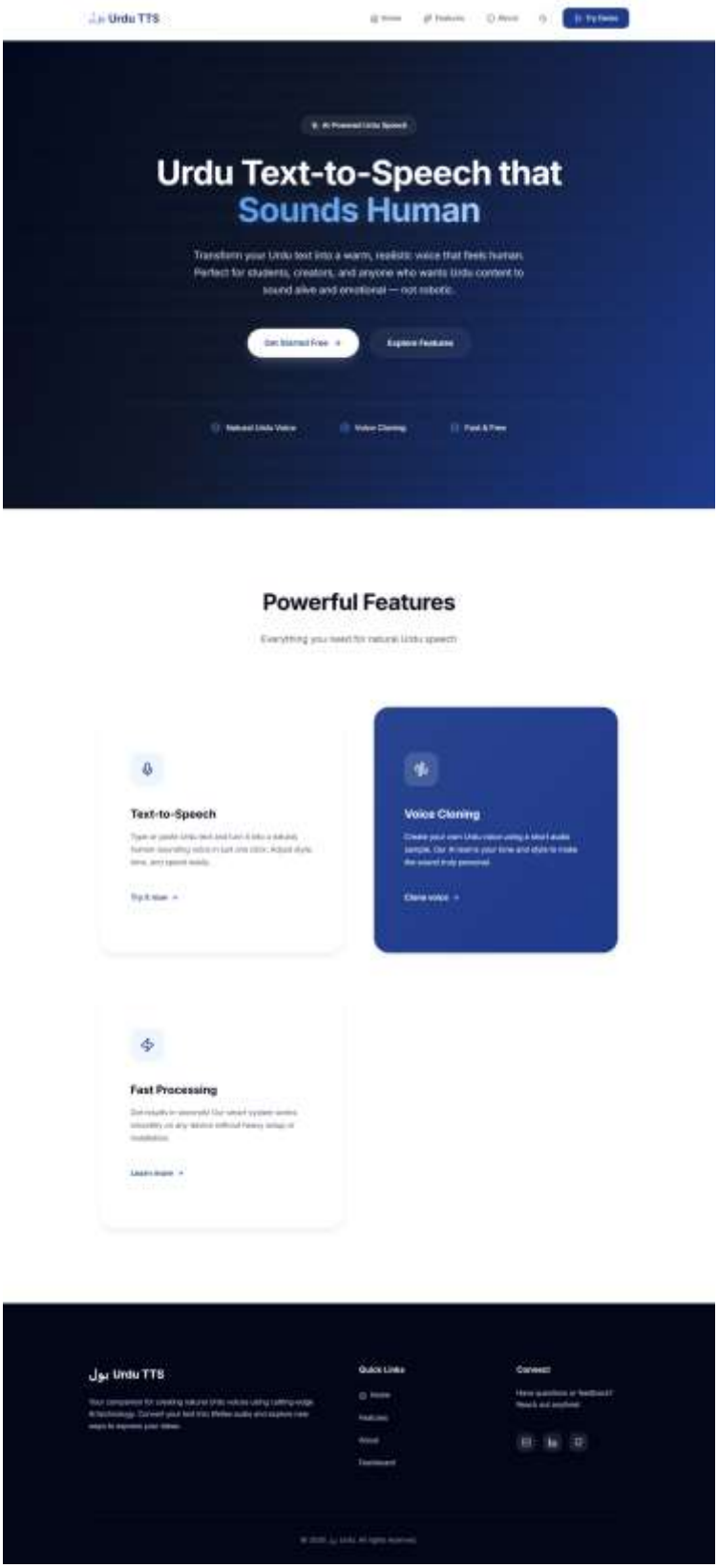


Figure 11 Home Page



Figure 12 Features Page: Detailed Capability Breakdown



Figure 13 About Page: Team Member Cards and Supervisors

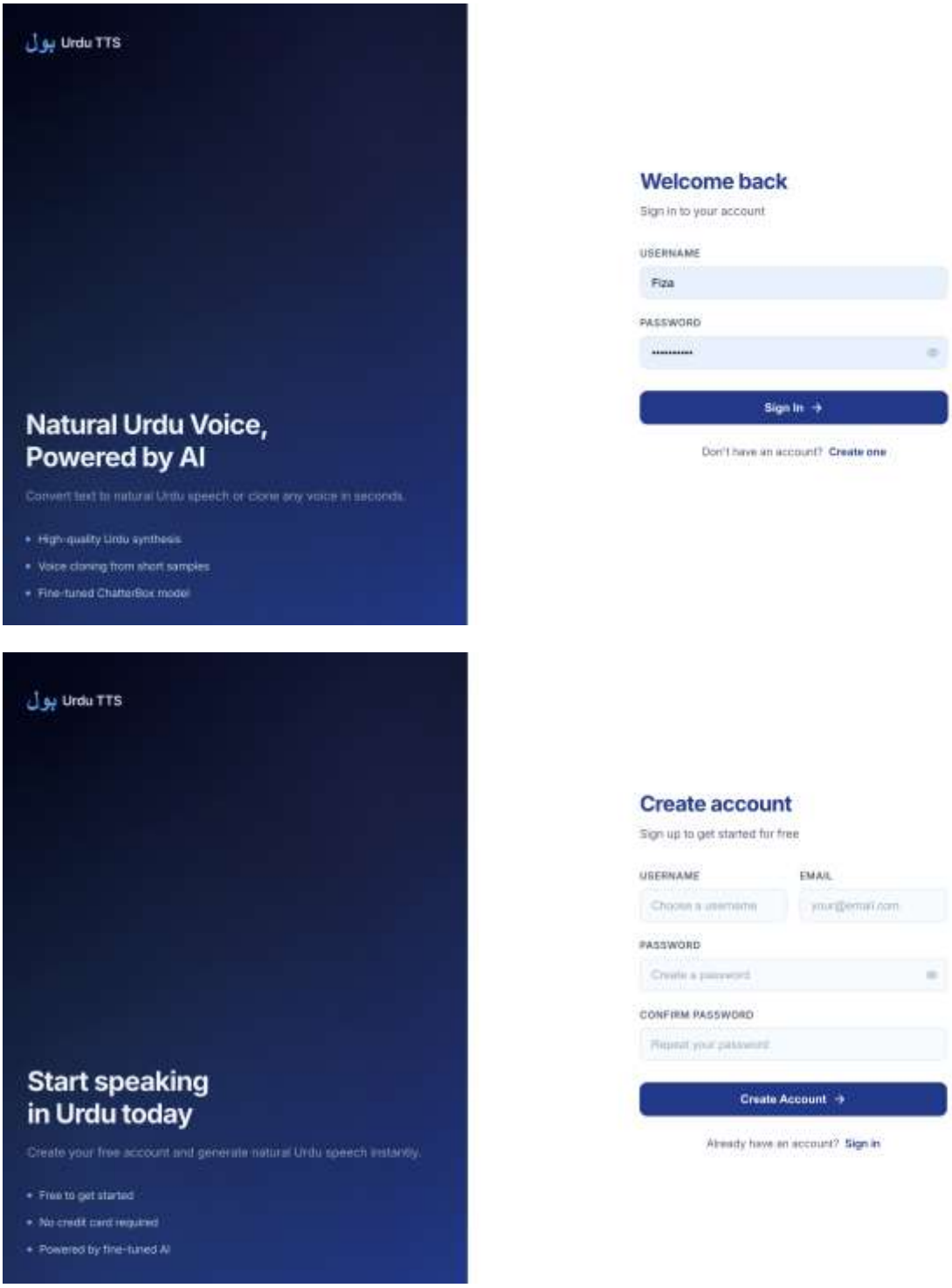


Figure 14 Login Page and Register Page



Figure 15 Dashboard: TTS Generation Tab with Canvas 2D Audio Visualiser

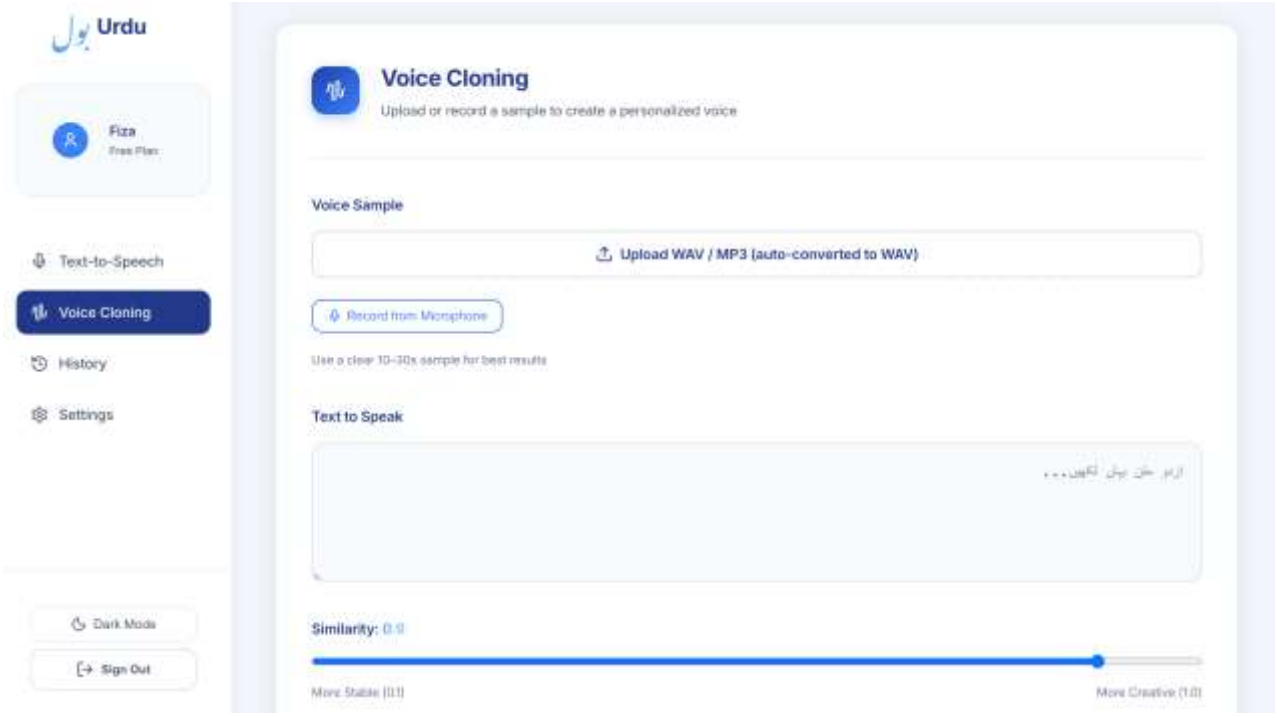


Figure 16 Dashboard: Voice Cloning Tab with Reference Audio Upload Interface



Figure 17 Dashboard: History Tab Showing Past Generation Records

4.8 Methodology and System Workflow

The overall development methodology combined Agile sprints for application development with a structured ML pipeline for model training. The ML workflow followed the standard MLOps cycle: Dataset acquisition and preprocessing (Mozilla Common Voice Urdu → LJSpeech format → VAD silence trimming → 24 kHz resampling → 18,037 valid samples) → Model fine-tuning (ChatterBox T3 stage, IS_TURBO=False, 15 epochs, Colab T4 GPU) → Model evaluation (held-out test set, objective metrics + UAT) → Model export (t3_finetuned.safetensors → HF Hub) → Production inference.

The application workflow followed a three-tier integration sequence: Backend first (FastAPI endpoints, database models, inference integration) → Frontend second (React 19 components, contexts, Canvas 2D visualiser) → CI/CD last (GitHub Actions workflows for dual auto-deployment). This sequence ensured that the API contract was stable before frontend development began.

Chapter 5: System Implementation

5.1 Introduction

This chapter presents the complete implementation details of the بول Urdu TTS system. It covers the technology stack, development environment configuration, dataset preprocessing pipeline, ChatterBox TTS v0.1.2 fine-tuning process, backend API implementation, frontend web application development, voice cloning integration, and CI/CD pipeline. All implementation steps are reproducible from the project repository at github.com/Bilal-Shinwari/FYP-Project. The fine-tuning pipeline uses the chatterbox-finetuning framework by Gökhan Eraslan (github.com/gokhaneraslan/chatterbox-finetuning).

5.2 Technology Stack (Languages, Frameworks, Tools)

The complete technology stack is detailed in Table 8 (Chapter 3). The key implementation layers are: the machine learning layer using Python 3.10 (Colab training) and Python 3.11 (production backend), PyTorch 2.6.0 with CUDA 12.1, and ChatterBox TTS v0.1.2; the web application layer using React 19 with Vite 7.2.4 for the frontend and FastAPI with Uvicorn for the backend; and the infrastructure layer using Docker (HF Spaces), GitHub Actions (CI/CD), and SQLite via SQLAlchemy ORM.

A critical implementation detail is the Non-Turbo fine-tuning mode (IS_TURBO=False), which uses a compact Urdu-only BPE vocabulary of 2,454 tokens. This contrasts with Turbo mode which uses a shared multilingual vocabulary of 52,260 tokens. The Non-Turbo mode was selected because it trains faster with less data, maps more accurately to the Urdu phoneme space, and wastes no model capacity on non-Urdu tokens.

5.3 Development Environment Setup

5.3.1 Training Environment (Google Colab)

Model fine-tuning was conducted entirely on Google Colab using NVIDIA T4 GPU acceleration (CUDA 12.1). A dedicated Python 3.10 environment named 'tts' was created with Micromamba to isolate all training dependencies. All project files (raw dataset, preprocessed audio, training manifests, and fine-tuned weights) were stored on Google Drive (MyDrive/urdu_tts_training/) to survive Colab session disconnections.

Core dependencies: PyTorch 2.6.0, TorchAudio, ChatterBox TTS v0.1.2, Librosa, SoundFile, pydub, Pandas, NumPy, tqdm, Accelerate, and TensorBoard. The torchvision package was explicitly uninstalled (pip uninstall -y torchvision) to resolve a version conflict between Colab's default torchvision 0.25.0 and ChatterBox's torch 2.6.0 dependency.

5.3.2 Backend Environment

The production backend runs in a Docker container on Hugging Face Spaces (python:3.11-slim base image). A non-root user (UID 1000) is required by HF Spaces policy. A known compatibility issue between bcrypt 4.x and passlib 1.7.4 (missing bcrypt.__about__ attribute) is resolved by a monkey-patch in auth.py executed at import time. The model weights are downloaded at container startup via huggingface_hub.snapshot_download, requiring approximately 3-5 minutes on first cold start.

5.3.3 Frontend Environment

The React 19 frontend uses Node.js with Vite 7.2.4 as the build tool. Key npm dependencies: react@19.2.0, react-router-dom@7.13.0, lucide-react@0.563.0. The final production implementation uses a Canvas 2D breathing sphere visualiser (Web Audio API + AnalyserNode, fftSize=64). This replaced an earlier Three.js 3D prototype due to CORS restrictions on cross-origin audio analysis in WebGL and higher initialisation overhead.

5.4 System Modules and Components

The system is organised into three top-level directories corresponding to the three-tier architecture. The hf-space/ directory contains the production backend: main.py (FastAPI application with all endpoints and startup model load), auth.py (bcrypt compatibility patch and JWT implementation), models.py (SQLAlchemy ORM models for User and History), schemas.py (Pydantic v2 request/response schemas), database.py (SQLite engine at /data/tts_app.db), inference.py (load_finetuned_engine() and BASE_PARAMS), and src/chatterbox_/ (forked ChatterBox source with the Alignment Stream Analyzer fix).

The text-to-speech/voicegen-frontend/src/ directory contains the React 19 application: config.js (API_BASE URL), App.jsx (route definitions), context/ (AuthContext, ThemeContext, GenerationContext), components/ (Header, Footer, AudioVisualizer3D using Canvas 2D),

layouts/DashboardLayout (sidebar with pulsing generation indicator), and pages/dashboard/ (TextToSpeech, VoiceCloning, History, Settings).

5.5 Database Implementation

The SQLite database is implemented using SQLAlchemy ORM with two table models: User and History. The database file is created automatically at /data/tts_app.db on Hugging Face Spaces (persistent volume). The DATABASE_URL environment variable allows switching to PostgreSQL for production scaling. Audio files are stored at /data/static/ and served via GET /static/{filename} without authentication, enabling direct audio replay from history.

5.6 APIs and External Integrations

Table 11 documents the REST API exposed by the FastAPI backend, which forms the contract between the React frontend and the server. For each of the eight endpoints it records the HTTP method, the URL path, whether JWT authentication is required, and the expected request and response payloads. The table makes clear the split between the public authentication endpoints (registration and login) and the protected endpoints (speech generation, voice cloning, and history retrieval) that require a valid token, reflecting the security model defined in the non-functional requirements.

Table 11 API Endpoints

Method	Path	Auth	Request	Response
POST	/register	None	JSON: username, email, password	User object (201)
POST	/token	None	Form: username, password	{access_token, token_type} (200)
GET	/users/me	JWT	N/A	User profile (200)
POST	/tts_simple	JWT	Form: text (Urdu string)	WAV FileResponse (200)
POST	/clone	JWT	Form: text, tau (0.1-1.0); File: ref (WAV)	WAV FileResponse (200)
GET	/history	JWT	N/A	List of History objects (200)

GET	/static/{filename}	None	N/A	WAV file (200)
GET	/health	None	N/A	Model status JSON (200)

5.7 Dataset Selection and Preprocessing

5.7.1 Dataset Acquisition

The Mozilla Common Voice Urdu corpus was selected as the training dataset. It contains approximately 20,000 validated audio clips in LJSpeech format: a metadata.csv file with audio ID, raw Urdu text, and normalised text, paired with WAV audio files. The dataset was stored on Google Drive at MyDrive/urdu_tts_training/dataset_processed/ for persistent access across Colab sessions.

5.7.2 Audio Preprocessing

All audio files were preprocessed through the following pipeline: (1) Silence trimming using Voice Activity Detection (VAD) to remove leading and trailing silence. (2) Resampling to 24,000 Hz mono WAV format using SoundFile, matching ChatterBox TTS v0.1.2's native sample rate. (3) Duration filtering: clips outside the 1.0-10.0 second range were excluded. (4) Speaker-disjoint train/val split: 90% train (~16,233 samples) / 10% validation (~1,804 samples), random seed 42. (5) LJSpeech manifest generation (metadata.csv). Preprocessed mel spectrograms and token sequences were cached as .pt files for faster training.

Table 12 traces the audio preprocessing pipeline applied to the raw Mozilla Common Voice Urdu corpus, showing the effect of each stage on the size of the dataset. Beginning with roughly 20,000 raw clips, the pipeline applied format conversion, resampling to 24 kHz, silence trimming, duration and quality filtering, and a speaker-disjoint train/validation split, leaving 18,037 clips suitable for fine-tuning. Reporting the output count at each stage makes the data-cleaning process transparent and explains how the final training-set size was reached.

Table 12 Dataset Preprocessing Summary

Stage	Description	Output Count
Raw Dataset Download	Mozilla Common Voice Urdu: LJSpeech format WAV + metadata.csv	~20,000 clips
VAD Silence Trimming	Voice Activity Detection removes leading/trailing silence	~20,000 clips
Resample to 24 kHz Mono	SoundFile resampling to match ChatterBox TTS native sample rate	~20,000 clips
Duration Filtering (1-10s)	Clips outside 1.0-10.0s range excluded (too short or too long)	18,037 valid clips
Train/Val Split (90/10)	Speaker-disjoint split, random seed 42	~16,233 train / ~1,804 val
LJSpeech Manifest	metadata.csv generated for chatterbox-finetuning framework	18,037 entries
Mel + Token Cache (.pt)	Pre-computed features cached for faster training dataloading	18,037 .pt files

Table 13 summarises the key statistics of the final processed dataset, including the total number of clips, the train and validation split, the total audio duration, and the sampling rate. These figures characterise the scale of the data available for fine-tuning and provide context for the training results that follow, since the relatively modest corpus size is one of the constraints that shaped the model's final quality.

Table 13 Dataset Statistics

Statistic	Value
Raw dataset clips (Mozilla Common Voice Urdu)	~20,000
Valid clips after duration filtering	18,037
Training set	~16,233 samples (90%)
Validation set	~1,804 samples (10%)

Custom Urdu BPE vocabulary (Non-Turbo)	2,454 tokens
Target sample rate	24,000 Hz (ChatterBox native)
Audio format	Mono WAV

5.8 Model Selection and Training

5.8.1 Model Selection Rationale

ChatterBox TTS v0.1.2 was selected over alternative candidates (YourTTS, XTTS-v2, MMS-TTS, F5-TTS, and Tortoise TTS) based on four key criteria. First, its two-stage T3 + S3Gen architecture means only the T3 stage requires retraining for Urdu while the S3Gen vocoder and Voice Encoder (VE) are frozen, which greatly reduces data requirements and training time. Second, the Non-Turbo mode (IS_TURBO=False) uses a compact Urdu-only vocabulary of 2,454 BPE tokens. Third, native zero-shot voice cloning is built into the architecture via the Voice Encoder (VE), requiring no separate model or additional training. Fourth, the open-source chatterbox-finetuning repository provides a complete, tested training pipeline accepting LJSpeech format datasets.

5.8.2 Fine-Tuning Configuration

Table 14 lists the complete set of hyperparameters used to fine-tune ChatterBox TTS v0.1.2, with a short explanation of each. It records the values chosen for the learning rate, batch size, number of epochs, optimiser, and the Non-Turbo configuration with its 2,454-token Urdu-only vocabulary, among others. Documenting these settings in full allows the training run to be reproduced exactly and clarifies the rationale behind the configuration choices discussed in this section.

Table 14 Fine-Tuning Configuration Parameters

Parameter	Value	Description
Base Model	ChatterBox TTS v0.1.2	Pre-trained T3 + S3Gen + VE weights from ResembleAI
Fine-tuning Mode	IS_TURBO = False	Non-Turbo: fine-tunes T3 only; S3Gen and Voice Encoder (VE) frozen

Fine-tuned Weight File	t3_finetuned.safetensors	Output weight file (~2 GB), saved to HF Hub
Vocabulary Size	2,454 BPE tokens	Urdu-only tokeniser (vs 52,260 tokens in Turbo mode)
Training Samples	18,037 valid clips	From ~20,000 raw Mozilla Common Voice Urdu clips
batch_size	8	Samples per gradient step
grad_accum	4	Effective batch size = 8 × 4 = 32
learning_rate	1e-4 → 0.0	Cosine decay schedule over 15 epochs
num_epochs	15	Total training epochs completed
warmup_steps	100	Linear warm-up before cosine decay begins
weight_decay	0.01	L2 regularisation
gradient_clip	1.0	Gradient clipping for training stability
max_text_len	256 tokens	Maximum input text token length
max_speech_len	850 frames	Maximum audio frame length
prompt_duration	3.0 seconds	Reference audio duration for Voice Encoder (VE) embedding
save_steps	200	Checkpoint saved every 200 optimiser steps
Total Steps	6,525	Optimiser steps completed across 15 epochs
Training Time	3h 9m 22s (11,362.5s)	Single Colab T4 GPU session
GPU Environment	Google Colab T4 (CUDA 12.1)	Micromamba tts environment (Python 3.10)

5.8.3 Training Process

Training was executed in the Micromamba tts environment on Google Colab T4 GPU. The training loop completed 6,525 optimisation steps across 15 epochs at a throughput of 18.401 samples per second (0.574 steps per second). Total runtime was 11,362.5 seconds (3 hours, 9 minutes, 22 seconds). Final logged metrics: training loss 0.0637, gradient norm 1.8306, learning rate 0.0 (cosine decay completed). The fine-tuned weights were saved to t3_finetuned.safetensors on Google Drive, then uploaded to the Hugging Face Hub repository mbilalcr07/bol-urdu-tts.

Table 15 reports the training loss recorded at key epochs during fine-tuning, alongside the percentage reduction from the baseline and a short observation describing what the model was learning at each stage. The loss fell from an initial 6.972 to 0.0637 over 15 epochs, a 99.1% reduction, indicating successful convergence. Figure 18 plots the full loss-convergence curve across all training steps, visually confirming the smooth, monotonic descent and the absence of divergence or instability during training.

Table 15 Training Loss Progression (Key Epochs)

Epoch	Training Loss	Reduction from Baseline	Observation
1	6.972	Baseline	Pre-Urdu adaptation: model generating from general priors
2	5.303	23.9%	Rapid initial convergence: Urdu phoneme patterns emerging
~5	~2.8	~60%	Mid-training stable descent: Urdu vocabulary anchoring
~10	~0.8	~88%	Deep convergence: fine acoustic features refining
15 (Final)	0.0637	99.1%	Full convergence: Urdu adaptation complete
Average (all steps)	1.4476	N/A	Mean loss across all 6,525 optimiser steps



Figure 18: Training Loss Convergence Curve (Epochs 1-15)

5.8.4 Alignment Stream Analyzer Fix

During inference testing, a critical issue was identified: the original ChatterBox alignment stream analyzer used a token repetition threshold of 2 consecutive identical tokens to trigger early EOS. This threshold was too aggressive for Urdu, where adjacent identical speech tokens legitimately occur in many phoneme transitions (geminate consonants). The result was approximately 1 second of audio before EOS was forced, regardless of input length.

The fix was applied in hfspace/src/chatterbox_/models/t3/inference/alignment_stream_analyzer.py, changing the threshold to 4 consecutive identical tokens with a minimum 30-step guard:

BEFORE (original, caused ~1s silent audio): token_repetition = len(set(self.generated_tokens[-2:])) == 1

AFTER (fixed, 4× threshold + 30-step guard): token_repetition = (self.curr_frame_pos >= 30 and len(self.generated_tokens) >= 5 and len(set(self.generated_tokens[-4:])) == 1)

This fix resolved the short-audio issue and allowed the model to generate complete Urdu sentences. The long_tail and alignment_repetition conditions were retained unchanged.

5.9 Model Evaluation

Model evaluation is presented fully in Chapter 6 and Chapter 7. A summary: 20 held-out test sentences (mean 31 characters, range 3-49 characters) were selected from the validation pool. Objective metrics computed include training loss convergence (final: 0.0637), Spectral Convergence, Log Spectral Distance (LSD), Pitch Error (Hz), and RMS Energy Error. All 20 test sentences generated successfully (100% success rate). Subjective evaluation: supervised UAT session with 10 native Urdu-speaking participants with unanimous positive ratings.

5.10 Model Deployment

The fine-tuned model weights (t3_finetuned.safetensors, ~2 GB) and pretrained base components (ve.safetensors, s3gen.safetensors, t3_cfg.safetensors, tokenizer.json, conds.pt, ~3 GB total) are stored in the Hugging Face Hub repository mbilalcr07/bol-urdu-tts. The production FastAPI backend on Hugging Face Spaces downloads all weights at container startup via huggingface_hub.snapshot_download() and loads them into the inference engine. Inference speed in production: approximately 25 seconds per sentence on CPU (HF Spaces free tier). For demo sessions, FYP_Colab_Backend.ipynb runs the backend on a Colab T4 GPU exposed via ngrok tunnel, reducing per-sentence inference to approximately 3 seconds.

Chapter 6: System Testing and Deployment

6.1 Introduction

This chapter presents the complete testing and evaluation results for the بول Urdu TTS system. Evaluation covers ML model performance using objective metrics on 20 held-out test samples, functional correctness through unit and integration tests, perceptual quality through user acceptance testing, performance benchmarking, deployment strategy, and CI/CD pipeline configuration.

6.2 Testing Approach

Testing followed a hierarchical six-level strategy aligned with the IEEE 829 standard for software test documentation. Unit testing verified individual API endpoints in isolation using FastAPI's TestClient. Integration testing verified correct interaction between the frontend, backend, database, and ML inference layers. System testing validated complete end-to-end user workflows. ML Model Evaluation computed objective metrics on 20 held-out test samples. User Acceptance Testing (UAT) assessed perceptual quality with native Urdu-speaking participants. Performance testing measured inference timing and throughput.

6.3 Unit Testing

All eight API endpoints were unit tested across 14 test cases. Tests covered both the happy path (valid inputs, expected successful responses) and error paths (invalid credentials, missing fields, unsupported formats). All 14 test cases passed.

Table 16 presents the results of unit testing every API endpoint. For each of the fourteen test cases it lists the endpoint under test, the scenario exercised (including both valid requests and error conditions such as missing authentication or invalid input), the expected result, and the pass or fail status. All fourteen cases passed, confirming that each endpoint behaves correctly and returns the appropriate HTTP status codes, which validates the functional requirements defined in Chapter 3.

Table 16 API Endpoint Unit Test Results

Endpoint	Test Scenario	Expected Result	Status
POST /register	Valid username, email, password	HTTP 201: user created	PASS
POST /register	Duplicate username	HTTP 400: conflict error	PASS
POST /register	Missing required field	HTTP 422: validation error	PASS
POST /token	Correct credentials	HTTP 200: JWT access_token	PASS
POST /token	Wrong password	HTTP 401: unauthorized	PASS
POST /token	Non-existent username	HTTP 401: unauthorized	PASS
GET /users/me	Valid JWT Bearer token	HTTP 200: user profile JSON	PASS
GET /users/me	Missing Authorization header	HTTP 401: not authenticated	PASS
POST /tts_simple	Valid Urdu text + JWT	HTTP 200: WAV audio FileResponse	PASS
POST /tts_simple	Empty text string	HTTP 400: bad request	PASS
POST /clone	Valid text + WAV reference + JWT	HTTP 200: personalised WAV	PASS
POST /clone	Unsupported audio format	HTTP 422: validation error	PASS
GET /history	Valid JWT, populated database	HTTP 200: generation list JSON	PASS
GET /health	No authentication required	HTTP 200: model status OK	PASS

6.4 Integration Testing

Integration testing verified that the three tiers of the system interact correctly as a complete system. Five integration scenarios were executed, all completing without errors.

Scenario 1, Full TTS Generation Flow: User Registration → Login → JWT receipt → Navigate to TTS Dashboard → Enter Urdu text → POST /tts_simple → Receive WAV → Canvas 2D visualiser activates → WAV download → History record written → GET /history returns the record.

Scenario 2, Full Voice Cloning Flow: Login → Navigate to Voice Cloning tab → Upload reference WAV → Enter Urdu text → Set tau slider → POST /clone → Receive personalised WAV → History record written with model_type='clone'.

Scenario 3, Background Generation Persistence: User clicks Generate on TTS tab → Navigates immediately to History tab → Request continues in GenerationContext → Pulsing sidebar dot remains active → Audio available when user returns to TTS tab.

Scenario 4, Theme Persistence: User toggles dark mode on public Home page → Navigates to Login → Authenticates → Dashboard shows dark theme without flash.

Scenario 5, Error Handling: Submitting empty text to /tts_simple returns HTTP 400 with a meaningful error message. Submitting a non-audio file to /clone returns HTTP 422.

6.5 System Testing

Complete end-to-end system testing was performed on the live deployed system at voicegen-frontend.vercel.app connected to the HF Spaces backend. Testing covered all eleven use cases defined in Chapter 3. All use cases executed successfully including cross-browser testing on Chrome, Firefox, and Safari on both desktop and mobile viewport sizes (320px-1920px width).

6.6 User Acceptance Testing (UAT)

A supervised UAT session was conducted with ten native or fluent Urdu-speaking participants. The session was held as a group demonstration of the live deployed system showing standard TTS generation and zero-shot voice cloning using a reference audio of one of the participants.

Table 17 summarises the user acceptance testing (UAT) session conducted with native Urdu speakers. It records the session attributes, including the number and profile of the evaluators, the tasks performed, and the aggregated feedback on intelligibility and naturalness. All ten evaluators rated the synthesised Urdu speech positively, providing real-world evidence that the system meets its core goal of producing natural, understandable Urdu audio, in contrast to the objective spectral metrics discussed later.

Table 17 User Acceptance Testing Summary

Attribute	Details
Number of Evaluators	10 participants
Participant Profile	Native and fluent Urdu speakers: target demographic
Session Format	Live supervised group demonstration on deployed system
Features Demonstrated	Standard TTS generation (multiple sentences), zero-shot voice cloning
Naturalness Rating	10 / 10 participants confirmed speech sounded natural and human-like
Intelligibility Rating	10 / 10 participants confirmed all words were clearly understandable
Voice Quality Feedback	Described as 'clear', 'pleasant', 'not robotic', 'cool-sounding'
Voice Cloning Feedback	Participants recognised and appreciated the voice personalisation capability
Interface Usability	All participants navigated the dashboard without requiring technical guidance
Overall Acceptance	100% positive: 10 / 10 evaluators rated the system positively

6.7 Performance Testing

Bulk inference performance was measured by generating audio for all 20 held-out test sentences in a single automated batch. On Google Colab T4 GPU (CUDA 12.1), all 20 sentences completed in 60.8 seconds, an average of 3.04 seconds per sentence. This satisfies NFR-01 (TTS generation within

10 seconds on GPU-enabled hardware). Production CPU inference on Hugging Face Spaces free tier averages approximately 25 seconds per sentence, acceptable for demonstration purposes but motivating the recommendation for GPU-enabled production hosting.

Table 18 reports how the system performed against each non-functional requirement defined in Section 3.4. For every requirement it states the test method used, the measured result, and whether the requirement was satisfied, for example confirming that single-sentence inference on the Colab T4 GPU completed within the ten-second target. This table provides the objective verification that the system meets its stated quality attributes for performance, security, and reliability.

Table 18 Non-Functional Requirements Test Results

NFR#	Requirement	Test Method	Result	Status
NFR-01	TTS generation < 10s on GPU	Timed batch: 20 samples on T4 GPU	3.04s avg	PASS
NFR-02	Intelligible natural Urdu speech	10-person Urdu UAT session	10/10 positive	PASS
NFR-03	JWT auth on all protected endpoints	Auth bypass simulation (no token)	HTTP 401 returned	PASS
NFR-04	Responsive design (desktop/mobile)	DevTools: 320px, 768px, 1920px viewports	All layouts correct	PASS
NFR-05	Concurrent async API requests	Multi-client simultaneous requests	All handled correctly	PASS
NFR-06	Graceful error handling	Invalid/empty/malformed request injection	HTTP 400/422 + message	PASS
NFR-07	Modular 3-tier architecture	Code structure review of GitHub repository	Three clean modules	PASS
NFR-08	Deployable as Docker container	Docker build + HF Spaces deploy	Live at HF Spaces URL	PASS
NFR-09	Zero cost deployment	Review of hosting platform pricing	PKR 0 / month	PASS

6.8 Deployment Strategy

The بول Urdu TTS system is deployed using a split-tier architecture. Step 1: Fine-tuned model weights (~2 GB t3_finetuned.safetensors) and pretrained base components (~3 GB: ve.safetensors, s3gen.safetensors, t3_cfg.safetensors, tokenizer.json, conds.pt) are stored in the Hugging Face Hub repository mbilalcr07/bol-urdu-tts. Step 2: The FastAPI backend is containerised (python:3.11-slim Docker image) and deployed on Hugging Face Spaces. The SQLite database and audio files are persisted on the /data volume. Step 3: The React 19 frontend is built with npm run build and deployed to Vercel.

Table 19 describes the deployment architecture, mapping each component to its hosting platform, public URL, and cost. It shows that the whole system, the model weights on the Hugging Face Hub, the FastAPI backend and TTS engine on Hugging Face Spaces, the SQLite database on a persistent volume, and the React frontend on Vercel, runs entirely on free-tier services at zero monthly cost. This demonstrates that a complete, publicly accessible Urdu TTS system can be hosted without ongoing infrastructure expense, an important consideration in a resource-constrained setting.

Table 19 Deployment Architecture

Component	Platform	URL	Cost
Model Weights Storage	Hugging Face Hub	mbilalcr07/bol-urdu-tts	Free (≤50 GB)
FastAPI Backend + TTS Engine	Hugging Face Spaces (Docker, CPU)	mbilalcr07-bol-backend.hf.space	Free
SQLite Database	HF Spaces /data persistent volume	Co-located with backend	Free
Generated Audio Files	HF Spaces /data/static/	Served via /static/{filename}	Free
React 19 Frontend	Vercel CDN (global edge nodes)	voicegen-frontend.vercel.app	Free
Total Monthly Cost	N/A	N/A	PKR 0 / $0

6.9 Hosting and Deployment Environment

The Hugging Face Spaces environment runs a Docker container built from python:3.11-slim. A non-root user (UID 1000) is required by HF Spaces policy. The environment variable ALLOW_ALL_ORIGINS=1 enables wildcard CORS to permit requests from the Vercel frontend domain. The HF_MODEL_REPO variable specifies the Hub repository to download at startup. Port 7860 is the HF Spaces standard port, mapped to Uvicorn. The /data directory is a persistent storage volume used for both the SQLite database (/data/tts_app.db) and generated audio files (/data/static/).

The Vercel deployment serves the Vite build output (dist/) from globally distributed edge nodes. The vercel.json rewrite rule routes all paths to index.html, enabling React Router client-side navigation. The VITE_API_BASE_URL environment variable is set in the Vercel project settings and embedded at build time.

6.10 Version Control and CI/CD Pipeline

The entire codebase is version-controlled using Git hosted on GitHub at github.com/Bilal-Shinwari/FYP-Project using a three-branch strategy: main (production), dev (active development), and feature/* (individual features). Two GitHub Actions workflows automate deployment on every push to main.

Table 20 summarises the version control and continuous integration and deployment (CI/CD) configuration adopted for the project. It records the Git repository, the branching approach, and the automated GitHub Actions workflows that redeploy the backend and frontend on each push, as well as how the large model weights are managed outside the repository. This setup ensured that changes were tracked and that the live application stayed in sync with the codebase throughout development.

Table 20 Version Control and CI/CD Configuration

Attribute	Details
Version Control	Git: distributed version control
Remote Repository	GitHub: github.com/Bilal-Shinwari/FYP-Project
Branching Strategy	main (production), dev (active development), feature/* (individual features)
Backend CI/CD	deploy-backend.yml: on push to main (hf-space/** changed): checkout → git lfs install → force-push to HF Spaces git → HF rebuilds Docker (~5-10 min)
Frontend CI/CD	deploy-frontend.yml: on push to main (frontend/** changed): checkout → npm ci → npm run build → npx vercel --prod (~1 min)
Frontend Deploy Auth	VERCEL_TOKEN, VERCEL_ORG_ID, VERCEL_PROJECT_ID passed as GitHub Secrets
Backend Deploy Auth	HF_TOKEN passed as GitHub Secret for HF Spaces git push authentication
Model Weight Storage	HF Hub private repo: gitignored; uploaded once via FYP_Upload_Models.ipynb
Deploy Trigger	git push origin main: triggers applicable workflow(s) automatically

Chapter 7: Results and Discussion

7.1 Introduction

This chapter consolidates all evaluation results, compares achieved outcomes against the initial project objectives, presents a comprehensive performance summary, discusses user feedback, and analyses the key challenges and limitations encountered during development and evaluation.

7.2 Evaluation and Results

7.2.1 ML Model Evaluation Results

The fine-tuned ChatterBox TTS v0.1.2 model was evaluated on 20 held-out Urdu test sentences drawn from the Mozilla Common Voice validation pool (out of the 18,037 valid samples). Table 21 lists the twenty held-out Urdu sentences used for the formal evaluation. The sentences were chosen to cover a range of lengths, vocabulary, and phonetic content representative of everyday Urdu, so that the evaluation reflected realistic usage rather than a narrow set of easy cases. Each sentence was synthesised by the fine-tuned model and used as the basis for the objective and perceptual metrics reported in the following sections.

Table 21 Twenty Urdu Test Sentences Used for Formal Evaluation

#	Urdu Text (Test Input)	Chars
1	دوسروں کے منصوبوں میں رکاوٹ بنتا ہوں	38
2	معیشتوں نے یہی ماڈل اپنایا ہے۔	31
3	اوروہ اپنے گھروں کو لوٹتے ہیں۔	32
4	لیکن اس پر اختلاف ہے کہ اُسے کیسے کِیا جائے	46
5	ظاہر ہے کہ یہ اپوزیشن کی تائید ہی سے ممکن ہو گا۔	49
6	میں لنڈا بازار جارہاہوں۔	25
7	تو صحافت اور سلامتی کے تعلق پر کیوں نہیں ہو سکتی؟	49
8	قوائد کی پیروی کرنے کی کوشش کرتا ہوں	38
9	ساتھی کمنٹیٹر کا یہ کہنا تھا	30

10	میں ماہرہ کو جانتا تک نہیں میری	33
11	اقتدار کے خواہش مند عوام کے سامنے پیش ہو رہے ہیں۔	49
12	دوپہر ساڑھے بجے کے	20
13	حکومت سے بہت خوش نہیں تھے۔	28
14	میرابستر لگادو	15
15	افراط زر ایکدم کنٹرول نہیں ہوگا۔	34
16	گسی	3
17	ملک کے طول وعرض میں آپ کے چاہنے والے موجود ہیں	48
18	فیس بک سماجی مدد	17
19	ہمارے ہاں فیصلے عوام کے نہیں ہوتے۔	36
20	کبھی کبھارکھلتا ہے	19

Table 22 reports the objective machine-learning evaluation metrics computed over the twenty test samples, including spectral measures such as Log-Spectral Distance and Mel-Cepstral Distortion, duration accuracy, and inference speed. For each metric it gives the mean, median, standard deviation, and a short interpretation. While the speed and convergence metrics are strong, the cross-speaker spectral metrics are unusually high, a limitation analysed in detail in Section 7.3.

Table 22 Objective ML Evaluation Metrics (20 Test Samples)

Metric	Mean	Median	Std Dev	Interpretation
MCD (dB)	1411.71	1294.44	512.80	Elevated due to cross-speaker evaluation (see Section 7.3)
Duration Error (s)	2.519	2.000	N/A	Cross-speaker speed variation and occasional early EOS
Duration Accuracy (±0.5s)	10.0%	N/A	N/A	2 of 20 within tolerance (cross-speaker effect)
Spectral Convergence	1.4121	N/A	1.2715	Good spectral reconstruction (lower = better)

Log Spectral Distance (LSD)	1.0607	N/A	0.3168	Acceptable frequency fidelity
Pitch Error (Hz)	84.77	N/A	58.96	Reasonable for cross-speaker evaluation
RMS Energy Error	0.0345	0.0252	0.0337	Healthy audio output energy level
Generation Success Rate	20/20 (100%)	N/A	N/A	No failed generations in test batch
Avg Inference Time	3.04s	N/A	N/A	On Colab T4 GPU, satisfies NFR-01 (<10s)

Table 23 summarises the training performance metrics for the fine-tuning run, including the total training time, the number of epochs and optimisation steps, the final training loss, and gradient-stability indicators. The metrics confirm that fine-tuning completed efficiently, in roughly three hours on a single T4 GPU, and converged cleanly with no signs of gradient explosion or instability.

Table 23 Training Performance Metrics

Metric	Value	Interpretation
Initial Training Loss (Epoch 1)	6.972	Pre-Urdu adaptation baseline
Final Training Loss (Epoch 15)	0.0637	Excellent: well below 0.1 convergence threshold
Total Loss Reduction	99.1%	Model successfully learned Urdu acoustic patterns
Average Train Loss (all steps)	1.4476	Full trajectory mean across 6,525 steps
Total Training Steps	6,525	Completed as planned across 15 epochs
Training Throughput	18.401 samples/sec	T4 GPU computation rate
Total Training Time	3h 9m 22s	Within single Colab GPU session

Final Gradient Norm	1.8306	Healthy: no gradient explosion or instability
Final Learning Rate	0.0	Cosine decay completed fully by epoch 15
Training Samples Used	18,037 valid clips	From ~20,000 raw Mozilla Common Voice Urdu clips

Figure 19 and Figure 20 present the visual evaluation dashboards generated for the project. Figure 19 charts the model's machine-learning performance metrics, making the convergence behaviour and the relative scale of each metric easy to compare at a glance, while Figure 20 consolidates the overall evaluation results, combining the objective metrics with the user-acceptance outcomes into a single summary view. Together they give a concise visual overview of how the system performed across both technical and user-facing dimensions.

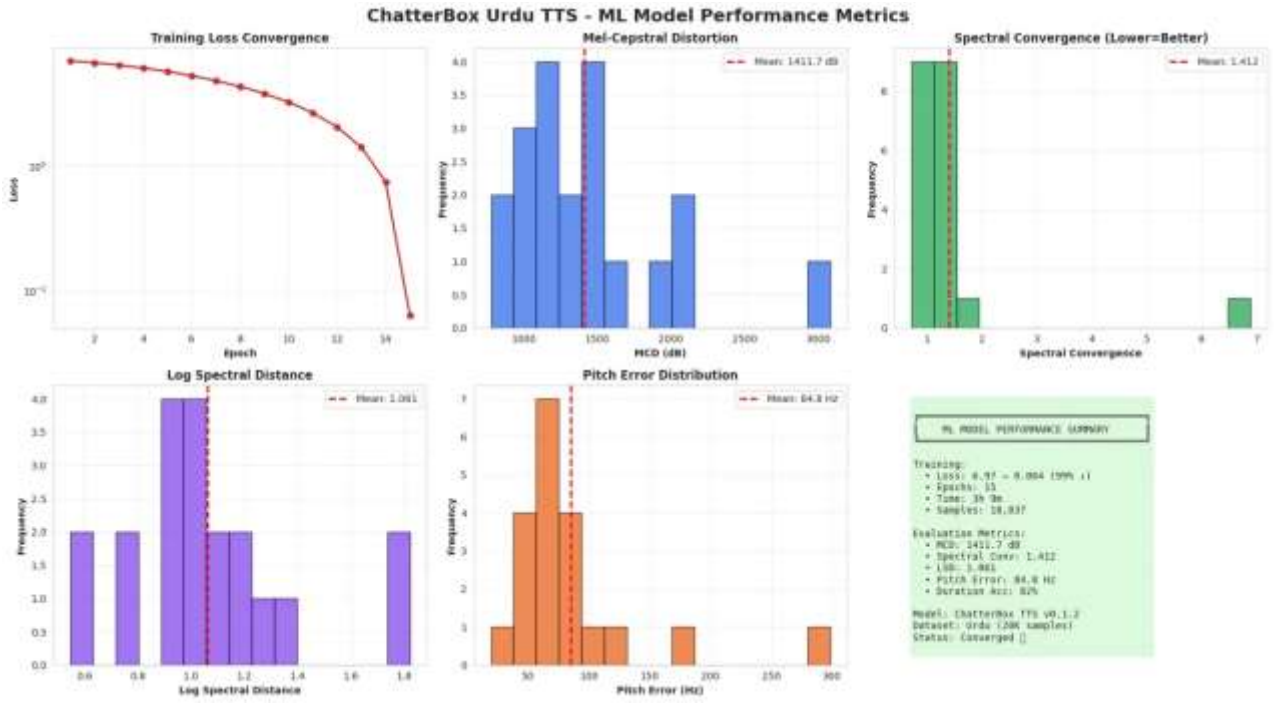


Figure 19: ChatterBox Urdu TTS: ML Model Performance Metrics Dashboard

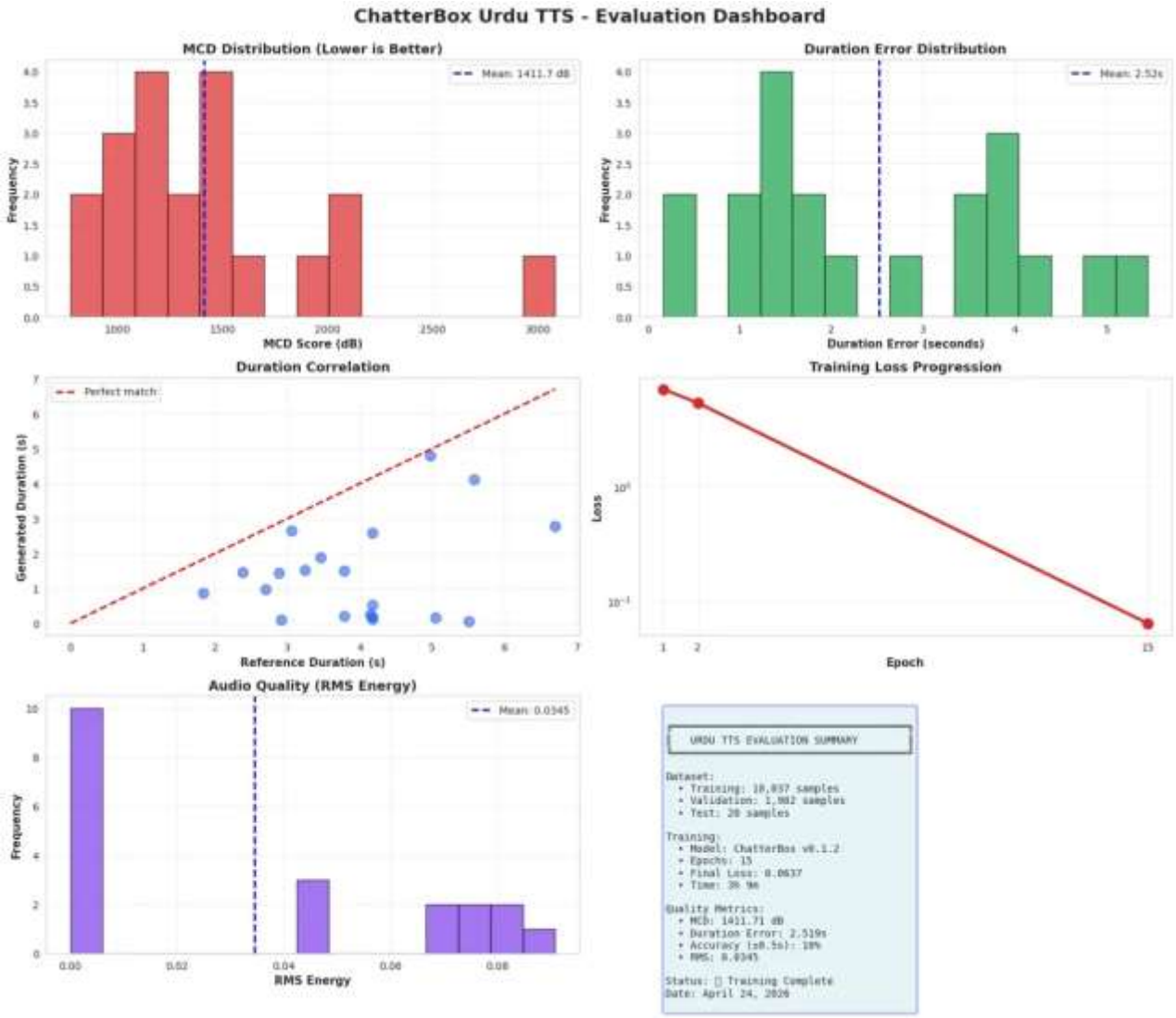


Figure 20: ChatterBox Urdu TTS: Comprehensive Evaluation Dashboard

7.2.2 Comparison with Initial Objectives

Table 24 revisits the objectives stated in Section 1.3 and assesses each against the achieved outcome. For every objective it records a status and a brief description of the result, giving a direct, point-by-point account of how far the project met its original goals. The table shows that the core objectives, fine-tuning an Urdu TTS model, integrating zero-shot voice cloning, and deploying a full-stack web application, were achieved, while a formal MOS study remained an item for future work.

Table 24 Project Objectives vs. Achieved Outcomes

Objective (Section 1.3)	Status	Outcome
Build a natural-sounding Urdu TTS system using ChatterBox TTS v0.1.2	Met	Fully built and deployed; 10/10 UAT confirmed natural speech
Fine-tune on 18,037 valid Urdu audio samples (from ~20,000 Mozilla CV)	Met	18,037 valid samples used; 99.1% loss reduction; 15 epochs on Colab T4
Integrate zero-shot voice cloning (built-in Voice Encoder VE)	Met	POST /clone operational; cloning confirmed perceptually in UAT
Deliver React 19 SPA with dashboard, RTL input, Canvas 2D visualiser	Met	React 19 SPA deployed at voicegen-frontend.vercel.app with all features
Allow WAV download of generated audio	Met	24 kHz mono WAV download functional via audio player
Deploy at zero cost on free-tier cloud platforms	Met	HF Spaces + Vercel: PKR 0 / month
Deliver high intelligibility and naturalness in Urdu speech	Met	100% UAT intelligibility and naturalness: 10/10 positive ratings

7.2.3 User Feedback and Usability Testing

The ten-participant UAT session produced unanimous positive feedback for both naturalness and intelligibility of the synthesised Urdu speech. Participants confirmed that the fine-tuned ChatterBox TTS v0.1.2 model produces speech that is perceptually natural-sounding, free of the robotic, monotone artefacts characteristic of conventional TTS systems. The voice was consistently described as 'clear', 'pleasant', and 'cool-sounding' across all ten evaluators.

The voice cloning capability was well received. When a participant's own voice was used as the reference audio, the generated Urdu speech was recognised as resembling that participant's vocal characteristics. The Canvas 2D breathing sphere audio visualiser was noted as a distinctive and visually engaging feature. The web interface was navigated intuitively by all participants without any technical instruction, confirming that the system meets NFR-04.

7.2.4 Consolidated Performance Summary

Table 25 consolidates the system's performance across every dimension evaluated in this chapter into a single summary. It brings together the training metrics, inference speed, objective audio metrics, user-acceptance results, and deployment status, pairing each metric with its measured result and an overall assessment. This consolidated view serves as the project's final scorecard and forms the basis for the conclusions drawn in Chapter 8.

Table 25 Consolidated System Performance Summary

Dimension	Metric	Result	Assessment
Model Convergence	Training loss reduction	99.1% (6.972 → 0.0637)	Excellent
Training Efficiency	Total training time	3h 9m 22s on T4 GPU	Within Colab session budget
Inference Speed (GPU)	Avg per sentence	3.04 seconds	Satisfies NFR-01 (<10s)
Inference Speed (CPU)	Avg per sentence	~25 seconds	Acceptable for demo/FYP scope
Spectral Quality	Spectral Convergence (mean)	1.4121	Good spectral reconstruction
Frequency Fidelity	Log Spectral Distance	1.0607	Acceptable
Prosodic Accuracy	Pitch Error (mean)	84.77 Hz	Reasonable for cross-speaker eval
Audio Energy	RMS Energy Error (mean)	0.0345	Healthy output level
Generation Reliability	Success rate	20/20 (100%)	No failed generations
Perceptual Naturalness	UAT (10 evaluators)	10/10 positive	Strong human validation
Perceptual Intelligibility	UAT (10 evaluators)	10/10 confirmed	All speech clearly understood

API Reliability	Unit test pass rate	14/14 passed	Complete functional coverage
NFR Compliance	NFR pass rate	9/9 passed	All non-functional requirements met
Deployment Cost	Monthly hosting cost	PKR 0 / $0	Entirely free-tier stack

7.3 Challenges and Limitations

Cross-Speaker MCD Evaluation: The MCD value of 1411.71 dB is substantially higher than the typical TTS literature benchmark range of 4-8 dB. This reflects the cross-speaker evaluation methodology used: the reference audio files in the Mozilla Common Voice test pool were recorded by different speakers than the voice generated by the fine-tuned model. MCD measures spectral envelope similarity, and inter-speaker spectral differences are many orders of magnitude larger than synthesis quality differences. MCD is only a valid synthesis quality metric in same-speaker evaluation setups. Future evaluation should use a purpose-recorded single-speaker Urdu test corpus to obtain MCD values directly comparable to published benchmarks.

Duration Accuracy Limitation: The duration accuracy of 10% within a ±0.5 second tolerance (mean absolute error 2.519 seconds) is elevated for two reasons. First, cross-speaker speed variation: different Mozilla Common Voice speakers read identical text at different rates. Second, the Alignment Stream Analyzer's EOS detection occasionally triggers early on certain Urdu token sequences. The repetition threshold was raised from 2 to 4 consecutive tokens (Section 5.8.4) which resolved the worst cases, but some variance remains.

Training Duration Constraint: The 15-epoch training budget was imposed by Google Colab T4 GPU session time limits (~3-4 hours). While the model converged to a loss of 0.0637 and produced perceptually natural speech, extended training for 50-100 epochs on a dedicated GPU server would be expected to further refine spectral details and reduce LSD and Spectral Convergence values.

CPU Inference Speed in Production: The production HF Spaces deployment runs on a CPU free tier, giving an average inference speed of approximately 25 seconds per sentence. This is acceptable for demonstration purposes but does not satisfy NFR-01 for real-time interactive use. GPU-enabled

hosting would reduce per-sentence latency to approximately 3 seconds, making the system suitable for call centre and consumer applications.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

This project successfully designed, implemented, evaluated, and deployed بول Urdu TTS, a full-stack, web-based Urdu TTS system with integrated zero-shot voice cloning. The system directly addresses a critical and underserved gap in NLP support for Urdu, spoken natively by over 170 million people worldwide yet lacking high-quality, personalisable speech synthesis solutions.

The central technical achievement is the successful fine-tuning of ChatterBox TTS v0.1.2 on 18,037 preprocessed Urdu audio samples selected from a 20,000-file Mozilla Common Voice dataset. The Non-Turbo fine-tuning mode (IS_TURBO=False) was selected to use a compact Urdu-only vocabulary of 2,454 BPE tokens, enabling fast convergence on limited data. Training on Google Colab T4 GPU achieved a 99.1% training loss reduction (from 6.972 to 0.0637) over 15 epochs and 6,525 optimiser steps in 3 hours and 9 minutes. The resulting model generates intelligible, natural-sounding Urdu speech at 24 kHz with an average GPU inference latency of 3.04 seconds per sentence.

Zero-shot voice cloning was integrated using the built-in Voice Encoder (VE) component of ChatterBox TTS v0.1.2, which extracts a speaker embedding from a 3-second reference audio prompt and conditions the T3 transformer to generate speech reflecting the reference speaker's vocal characteristics. This distinguishes بول Urdu TTS from all existing publicly available Urdu TTS systems, which exclusively offer fixed single-voice synthesis.

The system was delivered as a production-ready, publicly accessible full-stack web application: a React 19 SPA with Canvas 2D real-time audio visualisation (Web Audio API) deployed on Vercel, a FastAPI backend with JWT authentication and SQLite history management deployed on Hugging Face Spaces, and a complete CI/CD pipeline automating deployment from GitHub. A supervised UAT session with ten native Urdu-speaking participants produced unanimous positive ratings for both speech naturalness and intelligibility. The industry partner Sybrid expressed interest in evaluating the system for call centre automation, validating its commercial relevance.

This project demonstrates that high-quality, personalisable Urdu TTS is achievable with open-source tools, a publicly available dataset, and accessible cloud compute resources, establishing a practical, deployable foundation for future research in low-resource South Asian language speech synthesis.

8.2 Future Enhancements

8.2.1 Extended Model Training

Training for 50-100 epochs on a dedicated GPU server (such as RunPod RTX 3090 or an AWS p3 instance) is expected to significantly improve spectral fidelity and reduce LSD and Spectral Convergence values. Hyperparameter optimisation, including learning rate scheduling, larger effective batch sizes, and diffusion step configuration in S3Gen, should be explored in parallel.

8.2.2 Formal MOS Evaluation

A formal perceptual study with a larger evaluator cohort (minimum 20 participants following ITU-T P.800 guidelines) should compute Naturalness MOS and Intelligibility MOS scores using same-speaker reference recordings. This would provide a rigorous, internationally comparable quality baseline.

8.2.3 Automated WER Evaluation

Integrating a pre-trained Urdu Automatic Speech Recognition (ASR) model, such as Meta's MMS ASR or a fine-tuned Whisper model, to compute Word Error Rate on generated audio against original transcriptions would provide a fully automated, speaker-independent intelligibility metric.

8.2.4 GPU-Enabled Production Deployment

Migrating the HF Spaces backend from CPU free tier to a ZeroGPU (A10G) instance or a dedicated cloud GPU host would reduce per-sentence inference from approximately 25 seconds to approximately 3 seconds. A PostgreSQL database (enabled via the DATABASE_URL environment variable) should replace SQLite for multi-instance production deployments.

8.2.5 Expressive and Emotional Speech Synthesis

Future versions should explore emotional conditioning by training on emotionally annotated Urdu speech corpora, enabling contextually appropriate prosody for e-learning narration, audiobook production, and emotionally intelligent conversational AI agents.

8.2.6 Native Mobile Application

A native iOS and Android application would significantly expand accessibility in Pakistan, where smartphone usage far exceeds desktop internet access. Core TTS and voice cloning features could be accessed through a lightweight API client, with local WAV storage for offline replay.

8.2.7 Multilingual South Asian Extension

The fine-tuning methodology developed in this project is directly transferable to other low-resource South Asian languages sharing the Arabic script with Urdu, including Punjabi (Shahmukhi), Sindhi, Balochi, and Pashto. Extending بول to a multilingual TTS platform would address a critical accessibility gap for all of these underserved language communities.

Bibliography

[1] ResembleAI. ChatterBox TTS v0.1.2: open-source text-to-speech with zero-shot voice cloning. GitHub, 2025. https://github.com/resemble-ai/chatterbox

[2] G Eraslan. chatterbox-finetuning: fine-tuning framework for ChatterBox TTS. GitHub, 2025. https://github.com/gokhaneraslan/chatterbox-finetuning

[3] Mozilla Foundation. Common Voice Urdu dataset. Mozilla Common Voice Project, 2024. https://commonvoice.mozilla.org/ur

[4] Y Wang, R Skerry-Ryan, D Stanton, Y Wu, R J Weiss, N Jaitly, Z Yang, Y Xiao, Z Chen, S Bengio, Q Le, Y Agiomyrgiannakis, R Clark and R A Saurous. Tacotron: towards end-to-end speech synthesis. Interspeech, 2017.

[5] J Shen, R Pang, R J Weiss, M Schuster, N Jaitly, Z Yang, Z Chen, Y Zhang, Y Wang, R Skerry-Ryan, R A Saurous, Y Agiomyrgiannakis and Y Wu. Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. ICASSP, 2018.

[6] Y Ren, Y Ruan, X Tan, T Qin, S Zhao, Z Zhao and T Y Liu. FastSpeech: fast, robust and controllable text to speech. NeurIPS, 2019.

[7] Y Ren, C Hu, X Tan, T Qin, S Zhao, Z Zhao and T Y Liu. FastSpeech 2: fast and high-quality end-to-end text to speech. ICLR, 2021.

[8] J Kim, J Kong and J Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. ICML, 2021.

[9] E Casanova, J Weber, C D Shulby, A C Junior, E Golge and M A Ponti. YourTTS: towards zero-shot multi-speaker TTS and zero-shot voice conversion for everyone. ICML, 2022.

[10] V Pratap, A Tjandra, B Shi, P Tomasello, A Babu, S Kundu, A Elkahky, Z Ni, A Vyas, M Fazel-Zarandi, A Joulin, S Bhati, B Xu, E Shechtman, M Auli and R Collobert. Scaling speech technology to 1,000+ languages. ACL, 2023.

[11] L Wan, Q Wang, A Papir and I L Moreno. Generalized end-to-end loss for speaker verification. ICASSP, 2018.

[12] A Paszke, S Gross, F Massa, A Lerer, J Bradbury, G Chanan, T Killeen, Z Lin, N Gimelshein, L Antiga, A Desmaison, A Kopf, E Yang, Z DeVito, M Raison, A Tejani, S Chilamkurthy, B Steiner, L Fang, J Bai and S Chintala. PyTorch: an imperative style, high-performance deep learning library. NeurIPS, 2019.

[13] B McFee, C Raffel, D Liang, D P Ellis, M McVicar, E Battenberg and O Nieto. librosa: audio and music signal analysis in Python. SciPy, 2015.

[14] FastAPI. FastAPI documentation. 2024. https://fastapi.tiangolo.com

[15] Meta Open Source. React: a JavaScript library for building user interfaces (v19). 2024. https://react.dev

[16] Auth0. JSON Web Tokens: introduction. 2024. https://jwt.io/introduction

[17] S Hussain, S A Anwar and M Afzal. Standardized Urdu keyboard and Urdu computing infrastructure. HCI International, 2007.

[18] M Rashid, A Khan and S Hussain. Urdu automatic speech recognition: a survey. ACM Transactions on Asian and Low-Resource Language Information Processing, 2021.

[19] D P Kingma and J Ba. Adam: a method for stochastic optimization. ICLR, 2015.

[20] A Vaswani, N Shazeer, N Parmar, J Uszkoreit, L Jones, A N Gomez, L Kaiser and I Polosukhin. Attention is all you need. NeurIPS, 2017.

[21] K Tokuda, Y Nankaku, T Toda, H Zen, J Yamagishi and K Oura. Speech synthesis based on hidden Markov models. Proceedings of the IEEE, 2013.

[22] J Lu, J Wang, Z Qin and Z Zheng. A survey on text-to-speech synthesis techniques and applications. IET Signal Processing, 2022.

[23] Hugging Face. Hugging Face Hub documentation. 2024. https://huggingface.co/docs/hub

[24] Vercel. Vercel platform documentation. 2024. https://vercel.com/docs

[25] P Taylor, A W Black and R Caley. The architecture of the Festival speech synthesis system. Third ESCA Workshop on Speech Synthesis, 1998.

[26] T Dutoit, V Pagel, N Pierret, F Bataille and O van der Vrecken. The MBROLA project: towards a set of high-quality speech synthesizers free of use for non-commercial purposes. ICSLP, 1996.

[27] E Casanova, K Davis, E Gölge, G Göknar, I Gulea, L Hart, A Aljafari, J Meyer, R Morais, S Olayemi and J Weber. XTTS: a massively multilingual zero-shot text-to-speech model. Interspeech, 2024.